The InfoPharma Ordering System - Server Application

London Software House,

London, 2026

Summary

This is a Requirements Specification and High-Level Design Document, a proposal to the (Info Pharma Ltd) representatives: Mr. Lancaster (founder and CEO), Mr Walter (CTO) and Mr Khan (Head of IT). by the collective efforts of the London Software House.

Authors

Team 28 of the London Software House.

Distribution

(Info Pharma Ltd) representatives: Mr. Lancaster (founder and CEO), Mr Walter (CTO) and Mr Khan (Head of IT).

Table of Contents

The InfoPharma Ordering System - Server Application.... 1
London Software House,.... 1
London, 2026.... 1
Summary.... 1
This is a Requirements Specification and High-Level Design Document, a proposal to the (Info Pharma Ltd) representatives: Mr. Lancaster (founder and CEO), Mr Walter (CTO) and Mr Khan (Head of IT). by the collective efforts of the London Software House.... 1
Authors.... 1
Team 28 of the London Software House.... 1
Distribution.... 1
(Info Pharma Ltd) representatives: Mr. Lancaster (founder and CEO), Mr Walter (CTO) and Mr Khan (Head of IT).. 1
1. Preface.... 3
1.1 Purpose and the scope of the document (what it is and what is NOT).... 3
1.2 Intended Audience and Reading Suggestions.... 3
1.3 History of the document.... 3
2. Introduction.... 3
3. Glossary.... 4
4. User requirements.... 4
5. System requirements.... 4
5.1 System models.... 4
5.2 System models.... 4
5.2.1 UML use-case models.... 4
5.2.1.1.Use case diagram.... 4
5.2.1.2 Use case specifications.... 5
IPOS-SA Use Cases:.... 5
IPOS-SA-ACC:.... 6
IPOS-SA-CAT:.... 8
IPOS-SA-ORD:.... 12
IPOS-SA-RPT:.... 24
5.2.1.3 Prioritised List of Use Cases.... 33
6. Anticipated Evolution of System Requirements.... 35
6.1 Integration of Automated Payment Gateways and Online Banking.... 35
6.2 Automated Courier API Integration and Real-Time Tracking.... 35
7. IPOS High-Level Design.... 35
7.1 IPOS-SA.... 35
7.2 IPOS-CA.... 35
7.3 IPOS-PU.... 35
7.4 Software architecture.... 35
7.5 Design diagram of subsystem IPOS-SA.... 37
7.6 Source code (Java) for the interfaces of IPOS-SA.... 38
7.7 Testing the communications between subsystems.... 64
Interface Test Specifications – IPOS-SA (Team 28 – London Software House).... 121
8. Appendices.... 126
8.1 Initial Statement of requirements by the client.... 126
8.1.2 Hardware (minimal and optimal configuration).... 129
8.1.2.1 Minimal hardware configuration.... 129
8.1.2.2 Optimal hardware configuration.... 129
9. Credit List.... 129

1. Preface

1.1 Purpose and the scope of the document (what it is and what is NOT)

This is a Requirements Specification for one of the subsystems of InfoPharma Ltd.'s ordering system (IPOS), IPOS-SA and the High-Level Design document for the IPOS, which includes IPOS-SA and two other subsystems, IPOS-CA and IPOS-PU. The document covers the functional requirements of IPOS-SA and the high-level design of the entire IPOS, showing the IPOS architecture composed of three subsystems and the mechanisms of interaction between them. The document is provided in response to the initial statement of requirements by Mr Lancaster, InfoPharma Ltd, the Client.

The document explains what the subsystem shall do and how its subsystems might interact with one another, but it does not cover details about how the functionality of the IPOS-SA subsystem will be implemented. Various UML models are used together with fragments of software code to convey in more detail to system developers the expected behaviour of the IPOS-SA subsystem and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended Audience and Reading Suggestions

The Managing Director of Mr Lancaster, InfoPharma Ltd, the system analysts, architects and testers at the Centre for Software Development (CSD), London Software House.

Some of the sections are intended for the CSD team only, which is clearly stated in the document.

1.3 History of the document

This is a new document, v. 1.0.

2. Introduction

IPOS will allow InfoPharma Ltd. to automate its business processes in three interrelated areas: i) to maintain an electronic catalogue of pharmaceutical products and control stock availability, ii) to automate the operation of the merchant in dealing with its own customers, and iii) online sales of pharmaceutical goods to the public. By introducing IPOS-SA, the merchants will be able to access an online catalogue of goods, which will be updated every week, make an order and check the status of their orders.

The three areas of planned automation are expected to be covered in three subsystems: IPOS-SA, IPOS-CA and IPOS-PU, respectively. This document is focused on IPOS-SA. The other subsystems are covered in separate documents.

In addition to services offered to merchants, IPOS-SA allows managers to maintain an online catalogue of the medical goods provided and update the catalogue when needed. The system supports changing product details, deleting and adding new medicines, and managing stock availability.

IPOS-SA will also send automatic reminders and allow the suspension of the account to take place if no payment is received from the merchant. The generation of various reports is also taking place in this subsystem.

IPOS-SA interacts with the other subsystems, IPOS-CA and IPOS-PU.

3. Glossary

Actor	A human or an external system interacting with IPOS-SA, e.g. Merchant, Operation manager, Timer, etc.
CSD	Centre for Software Development
IPOS	InfoPharma Ltd.'s Ordering System
IPOS-SA	InfoPharma Ltd.'s Ordering System Server Application
IPOS-CA	InfoPharma Ltd.'s Ordering System Client Application
IPOS-PU	InfoPharma Ltd.'s Ordering System Public Online Portal
UML	Unified Modelling Language
Use case	A basic interaction of a system with an actor.

4. User requirements

This section is left intentionally blank.

5. System requirements

5.1 System models

This section provides a more detailed specification of the required functionality (known as system requirements) for IPOS-SA.

The intended audience is the system architects, designers and software developers responsible for the implementation of IPOS. Since we use UML, which requires a degree of specific expert knowledge, we do not expect this section to be immediately useful to Mr Lancaster or staff, but would like to encourage them to review and comment, if possible.

5.2 System models

5.2.1 UML use-case models

5.2.1.1.Use case diagram

The use case diagram for the IPOS-SA subsystem is shown below in Figure 1. In this diagram, the other subsystems of IPOS are treated as “external systems” with which IPOS-SA may interact.

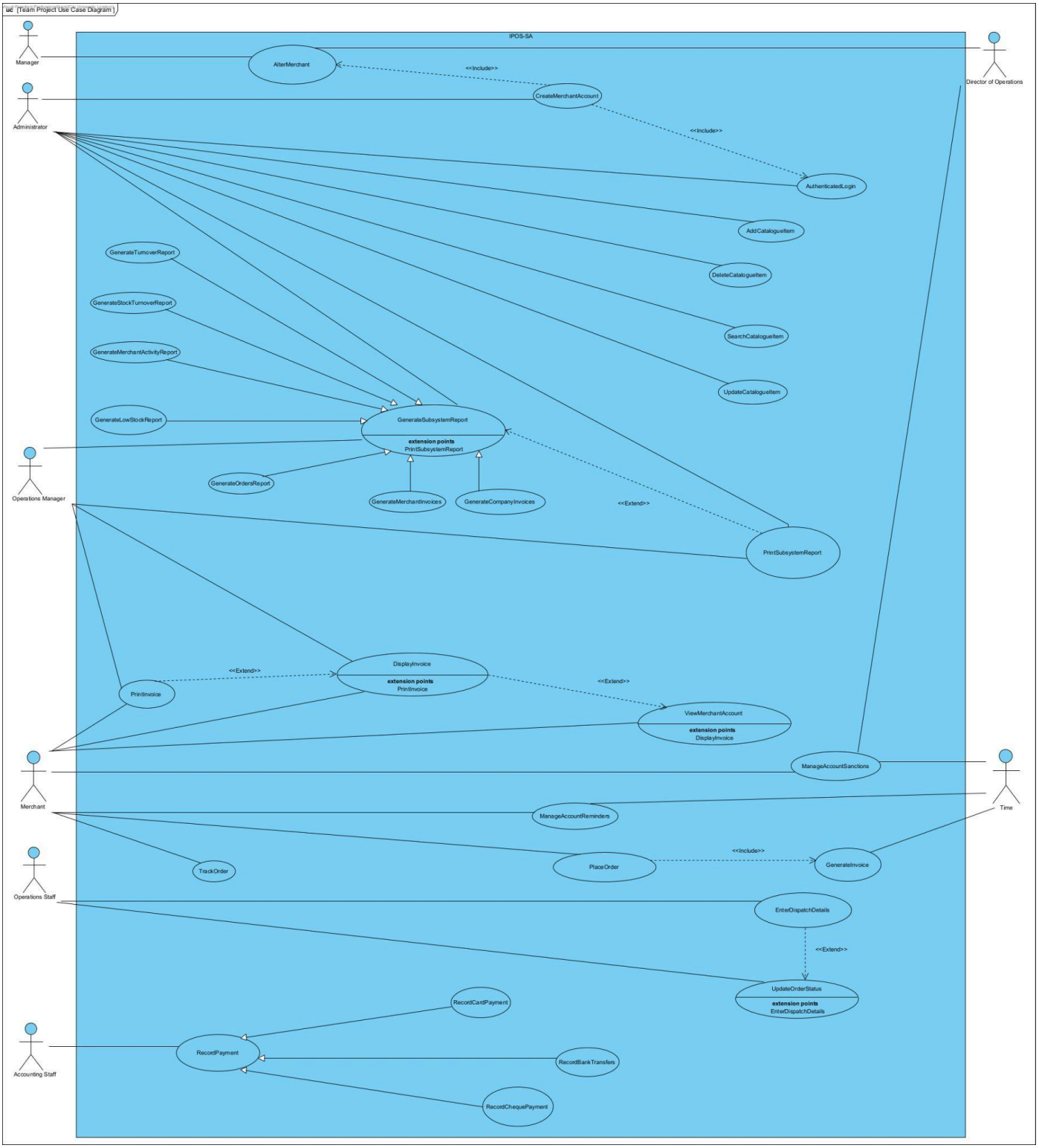

Figure 1. Use case diagram of IPOS-SA. The use cases cover the required functionality (see the initial statement of requirements). The actors represent the external entities (users or 3rd party systems) which interact with IPOS-SA. The other subsystems of IPOS are also modelled as actors.

5.2.1.2 Use case specifications

IPOS-SA Use Cases:

IPOS-SA-ACC:

Use case ID: 1	**Use case:** CreateMerchantAccount
Brief Description: An administrator sets up a new merchant account with contact details, credit limit, and a selected discount plan.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: 1. The administrator is authenticated and logged in. 2. The Merchant was not previously suspended.	
Main flow: Not required.	
Extensions: None	
Inclusions: AuthenticatedLogin	
Specialisation of: None	
Postconditions: 1. A new merchant record exists in the system database. 2. Account created with access to IPOS-SA-CAT AND IPOS-SA-ORD	
Alternative flows: MissingRequiredDetails (Account cannot be created if data is absent)	

Use case ID: 2	**Use case:** AuthenticatedLogin
Brief Description: The user account is activated and authenticated, then logs in.	
Primary actors: Administrator / Merchant / Manager	
Secondary actors: None.	

Preconditions: 1. IPOS-SA is operational. 2. User account exists.
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: None
Postconditions: 1. User is authenticated and logged in. 2. User role and privileges are established
Alternative flows: InvalidCredentials (Account cannot login if details are invalid or incorrect)

Use case ID: 3	**Use case:** AlterMerchant
Brief Description: The Director of Operations authorises a manager to manually restore an account from “in default” to “normal” or “suspended” or alter the credit limit / discount plan.	
Primary actors: Manager	
Secondary actors: Director of Operations	
Preconditions: 1. The Merchant account is currently “in default.” 2. The Director of Operations has authorised this change. 3. Payment delay has been longer than 30 days.	

Use case ID: 3	**Use case:** AlterMerchant
Brief Description: The Director of Operations authorises a manager to manually restore an account from “in default” to “normal” or “suspended” or alter the credit limit / discount plan.	
Primary actors: Manager	
Secondary actors: Director of Operations	
4. Manager is authenticated and logged in.	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: None	
Postconditions: 1. Account status is updated; merchants can resume ordering if the status is “normal.”	
Alternative flows: None	

IPOS-SA-CAT:

Use case ID: 4	**Use case:** AddCatalogueItem
Brief Description: An Administrator adds a new product to the electronic catalogue, including its description, unit price, and ID.	
Primary actors: Administrator	

Secondary actors: None.
Preconditions: 1. The administrator is authenticated and logged in. 2. The item is not already in the catalog.
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: None
Postconditions: 1. A new item is available for ordering.
Alternative flows: None

Use case ID: 5	**Use case:** DeleteCatalogueItem
Brief Description: An Administrator deletes a product from the electronic catalogue.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: 1. Administrator is authenticated and logged in. 2. The item is already in the catalog.	

Main flow: Not required.
Extensions: None
Inclusions: SearchCatalogueItem
Specialisation of: None
Postconditions: 1. Item is removed from the records and catalog 2. Item is no longer available to order..
Alternative flows: ItemNotFound

Use case ID: 6	**Use case:** SearchCatalogueItem
Brief Description: A user searches for a specific item using various searching criteria, item ID, key term etc).	
Primary actors: Merchant/Administrator	
Secondary actors: None	
Preconditions: 1. User is authenticated and logged in. 2. The item is already in the catalog.	
Main flow: Not required.	
Extensions: None	

Inclusions: None
Specialisation of: None
Postconditions: 1. The item is displayed or displays an error message showing item not found.
Alternative flows: ItemNotFound

Use case ID: 7	**Use case:** UpdateCatalogueItem
Brief Description: An Administrator updates details of an item, such as stock level, minimum stock parameter, description, etc.	
Primary actors: Administrator	
Secondary actors: None	
Preconditions: 1. The administrator is authenticated and logged in. 2. The item is already in the catalog.	
Main flow: Not required.	
Extensions: None	
Inclusions: SearchCatalogueItem	
Specialisation of: None	

Postconditions: 1. The item's information is correctly updated and displayed
Alternative flows: ItemNotFound

IPOS-SA-ORD:

Use case ID: 8	**Use case:** PlaceOrder
Brief Description: A Merchant accesses the electronic catalogue of pharmaceutical products and places an order by filling an order form.	
Primary actors: Merchant	
Secondary actors: None.	
Preconditions: 1. System is operational (i.e., stock is maintained and the order form functions correctly). 2. Merchant is authenticated and logged in. 3. Account status set as "normal". 4. A correct order form has been filled.	
Main flow: Not required.	
Extensions: None	
Inclusions: GenerateInvoice	
Specialisation of: None	

Postconditions: 1. Stock must be reduced to reflect units sold. 2. The sale is recorded
Alternative flows: InsufficientStock, OrderRejected

Use case ID: 9	**Use case:** GenerateInvoice
Brief Description: An invoice is generated and stored in the database for further reference, after an order has been made by a merchant.	
Primary actors: Time (instantly triggered by PlaceOrder)	
Secondary actors: None.	
Preconditions: 1. An order has been submitted..	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: None	
Postconditions: 1. Invoice must be stored in the system database for further reference	
Alternative flows: None	

Use case ID: 10	**Use case:** UpdateOrderStatus
Brief Description: The status of an order that has been made is changed to either: accepted, being processed, or dispatched.	
Primary actors: Operations staff	
Secondary actors: None.	
Preconditions: 1. The order must have been placed. 2. Staff are authenticated and logged in.	
Main flow: Not required.	
Extensions: EnterDispatchDetails	
Inclusions: None.	
Specialisation of: None	
Postconditions: 1. Status must be one of the 3 types specified. 2. Status visible to the merchant for tracking.	
Alternative flows: None	

Use case ID: 11	**Use case:** EnterDispatchDetails
Brief Description: Staff records the courier name, reference number, and expected delivery time for a dispatched order.	
Primary actors: Operations Staff	
Secondary actors: None.	
Preconditions: 1. Order status set to “dispatched” 2. Staff are authenticated and logged in.	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: None	
Postconditions: 1. Tracking information is recorded alongside the order.	
Alternative flows: None.	

Use case ID: 12	**Use case:** RecordPayment

Brief Description: Accounting staff enter identify a merchant account and start the process to record payments received to update balances on the account.
Primary actors: Accounting Staff
Secondary actors: None.
Preconditions: 1. Merchant has an authenticated account. 2. Staff are authenticated and logged in. 3. Valid payment method.
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: None
Postconditions: 1. Merchant account balance is updated 2. “suspended” accounts are restored to “normal” if payment recorded within 30 days.
Alternative flows: InvalidMerchantID

Use case ID: 12.1	**Use case:** RecordCardPayment
Brief Description: A specialised version of RecordPayment where the staff records details on transactions made using a credit or debit card.	

Primary actors: Accounting Staff
Secondary actors: None.
Preconditions (Inherited from RecordPayment) plus 4: 1. Merchant has an authenticated account. 2. Staff are authenticated and logged in. 3. Valid payment method 4. Card not expired.
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: RecordPayment
Postconditions (Inherited from RecordPayment): 1. Merchant account balance is updated 2. “suspended” accounts are restored to “normal” if payment recorded within 30 days.
Alternative flows (Inherited from RecordPayment) plus: CardDeclined

Use case ID: 12.2	**Use case:** RecordChequePayment
Brief Description: A specialised version of RecordPayment where the staff records details on transactions made using a bank cheque.	
Primary actors: Accounting Staff	
Secondary actors: None.	

Preconditions (Inherited from RecordPayment) plus 4:

1. Merchant has an authenticated account.
2. Staff are authenticated and logged in.
3. Valid payment method.
4. Cheque not expired

Main flow: Not required.

Extensions: None

Inclusions: None

Specialisation of: RecordPayment

Postconditions (Inherited from RecordPayment):

1. Merchant account balance is updated
2. "suspended" accounts are restored to "normal" if payment recorded within 30 days.

Alternative flows (Inherited from RecordPayment) plus:

InvalidChequeDetails

Use case ID: 12.3	**Use case:** RecordBankTransfers

Brief Description: A specialised version of RecordPayment where the staff records details on transactions made using a bank transfer.

Primary actors: Accounting Staff

Secondary actors: None.

Preconditions (Inherited from RecordPayment) plus 4:

1. Merchant has an authenticated account.
2. Staff are authenticated and logged in.
3. Valid payment method.
4. Authorized Bank

Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: RecordPayment
Postconditions (Inherited from RecordPayment): 1. Merchant account balance is updated 2. “suspended” accounts are restored to “normal” if payment recorded within 30 days.
Alternative flows (Inherited from RecordPayment) plus: TransferNotFound

Use case ID: 13	**Use case:** ManageAccountSanctions
Brief Description: System switches merchant accounts from “normal” to “suspended” to “in default” depending on lateness in payment.	
Primary actors: Time (via System Trigger)	
Secondary actors: Merchant	
Preconditions: 1. Merchant accounts must have overdue payments. 2. To switch status to “suspended”, Merchant payment must be 15 days late. 3. To switch status to “in default”, Merchant payment must be 30 days late.	
Main flow: Not required.	
Extensions: None	

Inclusions: None
Specialisation of:
Postconditions: 1. On screen reminder displayed for overdue payments, on each login 2. Account status changed depending on time period. 3. Orders can no longer be placed while account is “suspended” or “in default”
Alternative flows: None.

Use case ID: 14	**Use case:** ManageAccountReminders
Brief Description: System regularly monitors payment deadlines to trigger and display reminders to merchant accounts every time they log in.	
Primary actors: Time (via System Trigger)	
Secondary actors: Merchant	
Preconditions: 1. Merchant accounts must have overdue payments. 2. Overdue Payments must be less than 15 days overdue.	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: None	

Postconditions: 1. On screen reminder displayed for overdue payments, on each login
Alternative flows: None.

Use case ID: 15	**Use case:** ViewMerchantAccount
Brief Description: A Merchant is able to view their own account details, including: tracking order status, payments recorded, invoiced, outstanding unpaid balances etc.	
Primary actors: Merchant,	
Secondary actors: None.	
Preconditions: 1. Merchant is authenticated and logged in.	
Main flow: Not required.	
Extensions: DisplayInvoice	
Inclusions: None	
Specialisation of:	
Postconditions: 1. Account details must be displayed	
Alternative flows: None.	

Use case ID: 16	**Use case:** TrackOrder
Brief Description: A Merchant is able to track their order and it's status.	
Primary actors: Merchant,	
Secondary actors: None.	
Preconditions: 1. Merchant is authenticated and logged in. 2. An order exists within the database.	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of:	
Postconditions: 1. Order details must be displayed	
Alternative flows: None.	

Use case ID: 17	**Use case:** DisplayInvoice
Brief Description: A Merchant or Operations manager selects and displays an invoice that was generated when an order was placed.	
Primary actors: Merchant, Operations Manager	
Secondary actors: None.	
Preconditions: 1. Account is authenticated and logged in.. 2. The invoice was generated and stored previously for a specific order.	
Main flow: Not required.	
Extensions: PrintInvoice	
Inclusions: None	
Specialisation of: None	
Postconditions: 1. The invoice is displayed.	
Alternative flows: None.	

Use case ID: 18	**Use case:** PrintInvoice
Brief Description: A hard copy of a specific invoice is printed.	

Primary actors: Merchant, Operations Manager
Secondary actors: None.
Preconditions: 1. Account is authenticated and logged in. 2. Invoice was generated and stored previously for specific order. 3. Specific Invoice was selected to be printed.
Main flow: Not required.
Extensions: None.
Inclusions: None
Specialisation of: None
Postconditions: 1. Invoice is printed.
Alternative flows: None

IPOS-SA-RPT:

Use case ID: 19	**Use case:** GenerateSubsystemReport
Brief Description: Authorised administrators or managers initiate a request to search and gather business data from the database to be displayed as a report.	
Primary actors: Administrator, Operations Manager	

Secondary actors: None.
Preconditions: 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational
Main flow: Not required.
Extensions: PrintSubsystemReport
Inclusions: None
Specialisation of: None
Postconditions: 1. The report is stored 2. The report is displayed.
Alternative flows: DataNotFound

Use case ID: 19.1	**Use case:** GenerateTurnoverReport
Brief Description: Generates a specialized report showing total turnover for a period of time in terms of quantities of goods sold and revenue received and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database	

3. User is authenticated and logged in. 4. System Operational
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: GenerateSubsystemReport
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound

Use case ID: 19.2	**Use case:** GenerateStockTurnoverReport
Brief Description: Generates a specialized report showing goods sold and newly received over a period of time and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational	
Main flow: Not required.	
Extensions: None	

Inclusions: None
Specialisation of: GenerateSubsystemReport
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound

Use case ID: 19.3	**Use case:** GenerateMerchantActivityReport
Brief Description: Generates a specialized report showing the activity of an individual merchant over a period of time, including contact details and an itemized list of every order featuring quantities, costs and discounts and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: GenerateSubsystemReport	

Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound

Use case ID: 19.4	**Use case:** GenerateLowStockReport
Brief Description: Generates a specialized report showing a list of all items in the catalogue which are below their set minimum stock levels and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: GenerateSubsystemReport	
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.	

Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound

Use case ID: 19.5	**Use case:** GenerateOrdersReport
Brief Description: Generates a specialized report showing a list of orders received from a particular merchant for a given period of time with the relevant ID, dates, value, payment status and totals and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: GenerateSubsystemReport	
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.	
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound	

Use case ID: 19.6	**Use case:** GenerateMerchantInvoices
Brief Description: Generates a specialized report showing a list of detailed invoices raised against a merchant over a period of time and displays it on screen.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational	
Main flow: Not required.	
Extensions: None	
Inclusions: None	
Specialisation of: GenerateSubsystemReport	
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.	
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound	

Use case ID: 19.7	**Use case:** GenerateCompanyInvoices
Brief Description: Generates a specialized report showing all invoices raised byInfoPharma Ltd against merchants for a given period and displays it on screen.	

Primary actors: Administrator, Operations Manager
Secondary actors: None.
Preconditions (Inherited from GenerateSubsystemReport): 1. Appropriate data is searched for and gathered 2. All data exists with the database 3. User is authenticated and logged in. 4. System Operational
Main flow: Not required.
Extensions: None
Inclusions: None
Specialisation of: GenerateSubsystemReport
Postconditions (Inherited from GenerateSubsystemReport): 1. The report is stored 2. The report is displayed.
Alternative flows (Inherited from GenerateSubsystemReport): DataNotFound

Use case ID: 20	**Use case:** PrintSubsystemReport
Brief Description: Admins/ Managers can print a hard copy of a selected report.	
Primary actors: Administrator, Operations Manager	
Secondary actors: None.	

Preconditions:

1. User is authenticated and logged in.
2. User wishes to print a report.
3. The report exists.

Main flow: Not required.

Extensions: None.

Inclusions: None

Specialisation of: None

Postconditions:

1. The report is printed.

Alternative flows:

None.

5.2.1.3 Prioritised List of Use Cases

The list below prioritises the use cases included in the use case model according to their importance for the IPOS-SA. CSD management will use the assigned priorities to decide on the order of implementing the functionality specified in the use cases according to the resources available: the ***essential*** use cases will be implemented first, then the use cases ranked as ***important*** will follow. The use cases ranked as ***good-to-have*** will be implemented last, provided the resources (manpower and time) are available to complete their implementation.

Use case ID	Use case name	UC priority	Justification
1	CreateMerchantAccount	Essential	A core functionality as every merchant needs an account to make orders
2	AuthenticatedLogin	Essential	Similar justification to the first UC, but after account creation, this would be used to interact with IPOS
3	AlterMerchant	Important	This allows us to notify merchants about their credit limits
4	AddCatalogueItem	Essential	We need to add items to the catalogue, so customers can see the listing
5	DeleteCatalogueItem	Good-to-have	We may need to remove obsolete items from the listing
6	SearchCatalogueItem	Important	For convenience of use, to help customers find the desired items faster
7	UpdateCatalogueItem	Essential	We need to update the stock number, price or other details about the item to keep it up to date
8	PlaceOrder	Essential	The primary function of the IPOS-SA
9	GenerateInvoice	Essential	Creating the record of the financial operation of an order
10	UpdateOrderStatus	Important	Critical feature to update the status of already placed orders
11	EnterDispatchDetails	Important	Provides order with correct shipment details that are generated after the order placement
12	RecordPayment	Essential	Critical requirement as it would be necessary for creation of the reports in the future
13	RecordCardPayment	Essential	Similar justification to RecordPayment
14	RecordChequePayment	Important	Similar justification to

			RecordPayment, but less important as cheques are not used often
15	RecordBankTransfers	Important	Similar justification to RecordPayment, but less important as bank transfers are not used that often
16	ManageAccountSanctions	Important	If the merchant exceeds the limits, we need to impose sanctions on their account to limit stock lost without pay, incentivises them to pay back.
17	ManageAccountReminders	Important	Helps to keep merchants engaged and help the sales
18	ViewMerchantAccount	Important	Allows merchants to check the current account details
19	TrackOrder	Important	Allows merchants to check the status of the order
20	DisplayInvoice	Good-to-have	Allows merchants and staff to view invoices
21	PrintInvoice	Good-to-have	Allows merchants and staff to print invoices
22	GenerateSubsystemReport	Important	The core functionality that is used for other reporting functionality
23	GenerateTurnoverReport	Good-to-have	Core metric to understand quantities sold and revenue received
24	GenerateStockTurnoverReport	Good-to-have	Similar to the GenerateTurnoverReport but for stock imports/exports
25	GenerateMerchantActivityReport	Good-to-have	Useful to see the operation of the individual merchant
26	GenerateLowStockReport	Important	Helpful to the manager not let the stock shortages occur
27	GenerateOrdersReport	Good-to-have	Helpful to include in other types of reports
28	GenerateMerchantInvoices	Good-to-have	Useful for any financial disputes with the merchant
29	GenerateCompanyInvoices	Good-to-have	Helpful for accounting
30	PrintSubsystemReport	Good-to-have	Printing the report on paper is low priority requirement as most would be used as electronic documents

6. Anticipated Evolution of System Requirements

We expect that IPOS-SA will be developed and deployed quickly. The operational experience from merchants and staff will inform the future decisions about the necessary changes. The envisaged likely changes to various parts of IPOS-SA are summarised below.

6.1 Integration of Automated Payment Gateways and Online Banking

Currently, the prototype relies on the manual entry of payment details (such as bank transfers, cheques, and card payments) by the accounting staff at InfoPharma Ltd (Use Cases 12.1, 12.2, and 12.3). As the system scales, a primary evolution is the direct integration of IPOS-SA with online banking APIs and secure B2B payment gateways. This evolution will allow the system to automatically process transactions and dynamically update merchant balances without any human(staff) involvment into hte process. This addition will automate the lifting of account sanctions, significantly reducing the administrative burden on the SA staff.

6.2 Automated Courier API Integration and Real-Time Tracking

In the current system design, operations staff are required to manually enter dispatch details, including the courier name, reference number, and expected delivery time, once an order is physically shipped (Use Case 11: EnterDispatchDetails). A planned evolution for the system is to integrate directly with third-party logistics services and courier APIs (e.g., Royal Mail). This enhancement will allow IPOS-SA to automatically generate shipping labels and automatically update the order status to "Dispatched" upon sending

7. IPOS High-Level Design

This section defines the high-level design of the entire IPOS software, which includes its architecture - the subsystems which it is made up of, and the communication interfaces between the subsystems. The subsystems envisaged are:

7.1 IPOS-SA

This subsystem will allow members of the public to access the electronic catalogue and place orders with Info Pharma Ltd.

7.2 IPOS-CA

This subsystem will allow merchants and members of staff at Info Pharma Ltd to create an account in IPOS and manage it.

7.3 IPOS-PU

7.4 Software architecture

The architecture of IPOS as a UML component diagram, developed with Visual Paradigm, Professional Edition, is shown on ***Figure 2***.

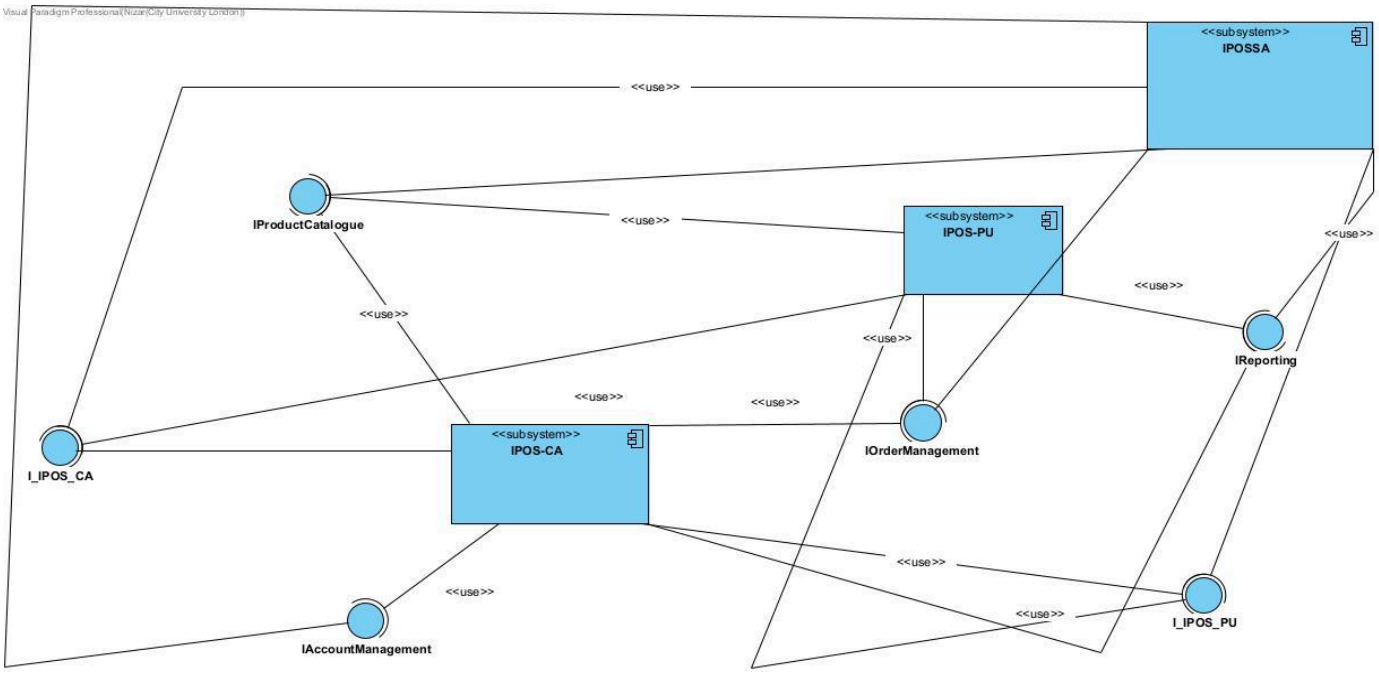

Figure 2. *IPOS-SA architecture as a UML component diagram. Each subsystem is modelled as a component with stereotype <<subsystem>>. The communication between the subsystems is achieved via a set of assembly connectors (i.e., a pair of "provided" and "required" interfaces referring to the API offered by the respective subsystems. IProductCatalogue, IReporting, IOrderManagement and IAccountManagement) are provided by IPOS-SA, I_IPOS_CA and I_IPOS_PU, respectively and the APIs are used to access the functionality offered by one of the subsystems by the other two subsystems.*

Figure 2 hides the API details. Using the "presentation options" of the Visual Paradigm, one can switch to "class" notations and disclose full interface details, as is illustrated on ***Figure 3***.

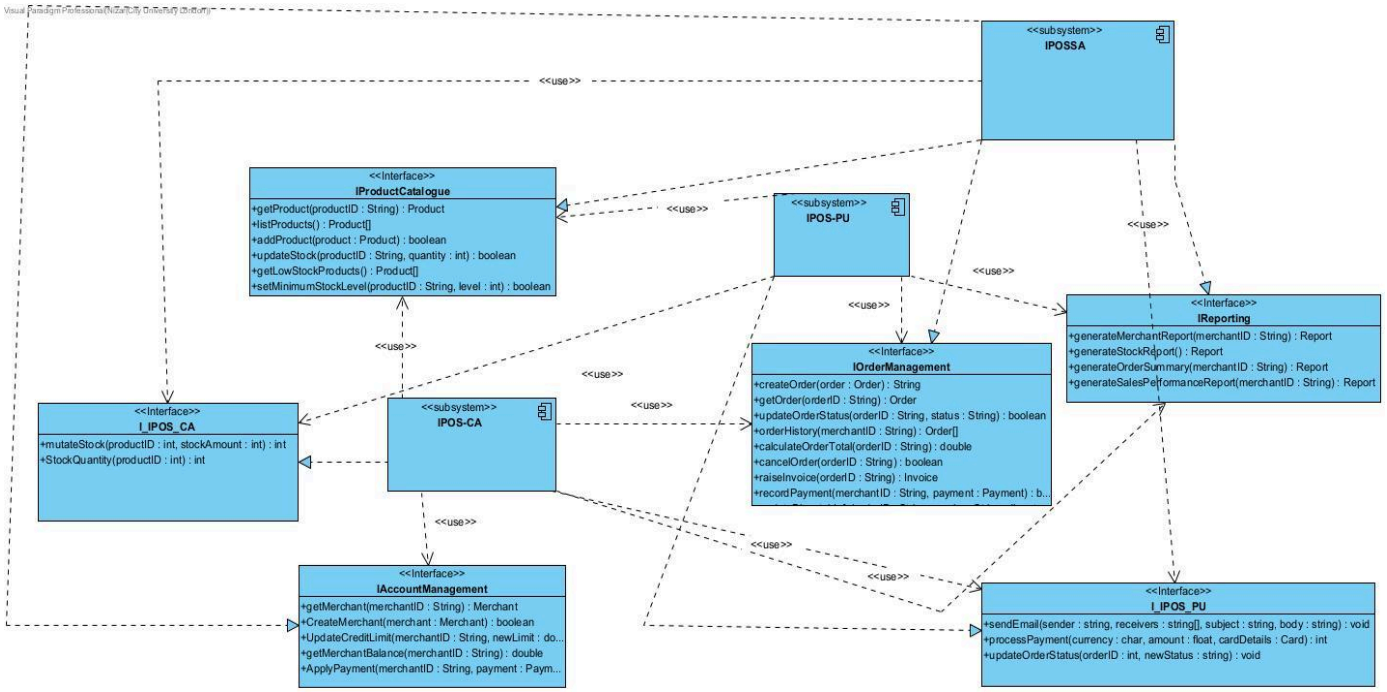

Figure 3. *Now the methods defined in interfaces are explicitly shown with their full signatures. These are plausible public methods via which the functionality of respective subsystems can be accessed. Note that changing the presentation of interfaces, changed the relationships of the interfaces, too: the subsystems responsible to implement the interface has a "realisation" relationship with it, which implies that the respective subsystem will contain an implementation class realising the interface. The subsystems using the API (i.e. for which the API is a "required" interface) now would have a dependency relationship with the interface (<<usage>> icon in Visual Paradigm should be used in the construction of the component diagram).*

The IPOS architecture includes three subsystems and shows the interfaces necessary for these subsystems to interact with each other.

Commentary

The diagrams shown in Figure 2 and Figure 3 are developed under the assumption that each subsystem can have multiple "provided" interfaces, and that the same provided interface will be used by the other two subsystems (for them this interface will be a "required" interface). This assumption does not necessarily hold true always. We can easily envisage cases where each subsystem may offer dedicated APIs to each of the other two subsystems, e.g., to achieve greater efficiency of the interaction, or simply to restrict the access by other subsystems to those parts of functionality which the respective subsystem may need (while access to other part of functionality may be blocked). For instance, IPOS-CA may offer access to merchant details by IPOS-SA via a dedicated interface and disallow IPOS-PU access to its functionality. Such an arrangement would seem plausible, as merchants and managers (using IPOS-SA) need access to the account details (held in IPOS-CA). Such access by IPOS-PU, used by the public, however, does not seem necessary. In fact, allowing public access to confidential merchant account information may be illegal.

7.5 Design diagram of subsystem IPOS-SA

Next, we elaborate on the high-level design of IPOS-SA and provide details on the interfaces that this subsystem needs, and on the implementation of the subsystem's required interface.

The class diagram of the IPOS-SA subsystem is shown in ***Figure 4***.

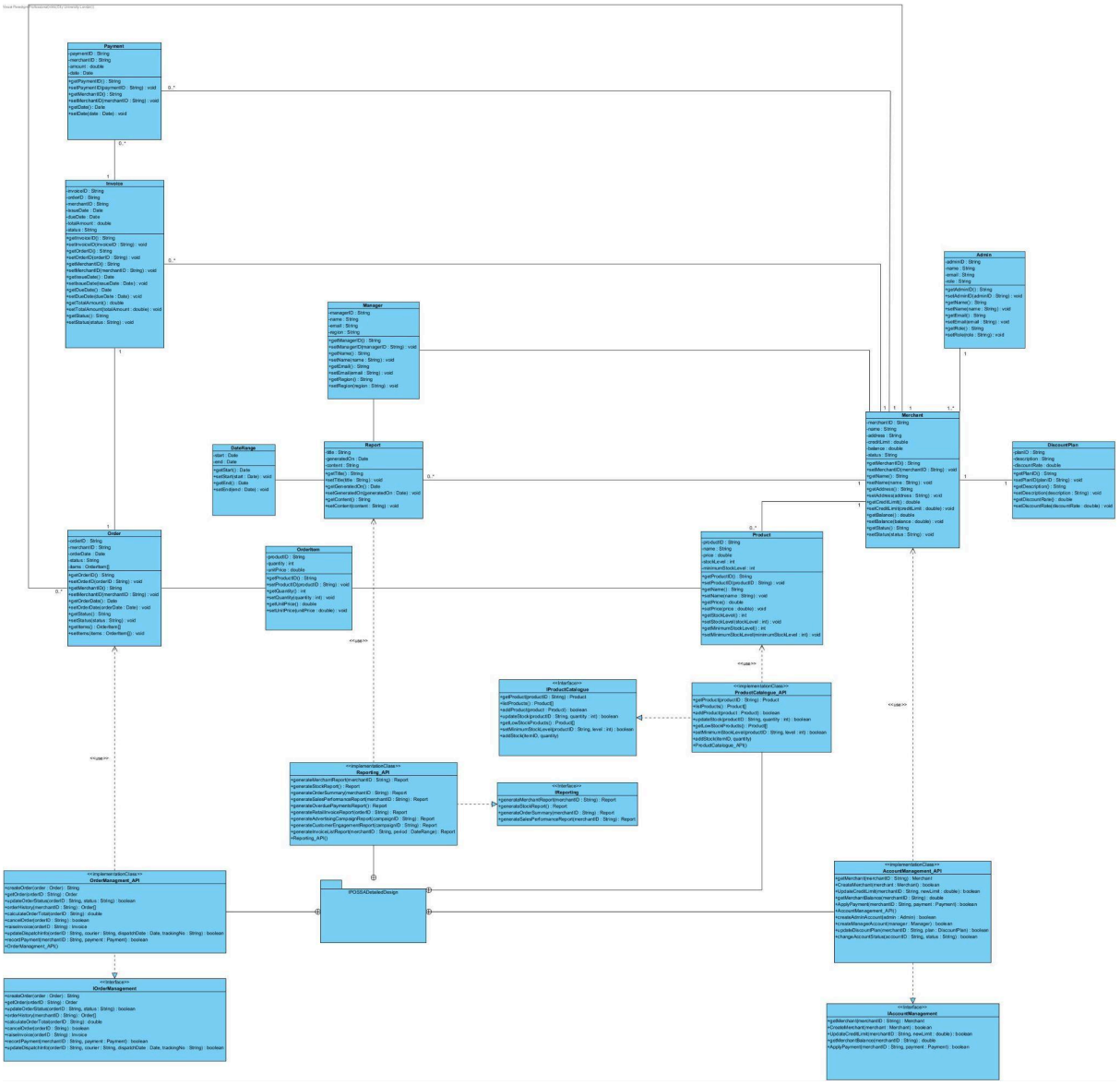

Figure 4. *High-Level design of IPOS-SA.*

Commentary

The diagram shows the interfaces that IPOS-SA depends on (which already appeared in full detail in Figure 3), and a minimal version of the implementation class IPOS_SA_API of the interface I_IPOS_SA, together with the realisation relationship between the provided interface and the implementation class. The diagram also makes it clear that the implementation class is part of the IPOS_SA_DetailedModel package: the containment relationship between the package and the implementation class shows that the implementation class belongs to the package.

At this stage, the package IPOS_SA_DetailedModel is just a placeholder. Its content – classes, interfaces, enumerations, etc., which together will deliver the functionality of the IPOS-SA subsystem – will be developed later, as part of the detailed design and implementation of IPOS-SA, which are yet to take place. The diagram also shows the required interfaces I_IPOS_CA and I_IPOS_PU, which we saw in Figure 3. The dependency relationship between the IPOS_SA_DetailedModel package and the two required interfaces indicates that some of the classes in the package IPOS_SA_DetailedModel may/will use the required interfaces (i.e., will be making calls of the methods defined in the interfaces) to access the functionality of the respective subsystems, which in turn will be providing an implementation for the methods of the required interfaces, as required.

The class diagram is developed under the assumption that IPOS-SA will need to access both subsystems. A similar diagram for the other subsystems will follow a similar pattern, but as discussed above, access to only some of the other subsystems may be sufficient. For instance, it is unlikely that IPOS-CA will need to access IPOS-PU directly, as online sales are typically "propagated" from the platform to the client inventory rather than the client polling the platform. If this is indeed the case, then the interface I_IPOS_PU may not be necessary in the class diagram of IPOS-CA.

7.6 Source code (Java) for the interfaces of IPOS-SA

Now that we have defined all interfaces and the implementation class CoreAPI, source code could be derived (e.g. Java source code) for the interfaces and the implementation classes shown in *Figure 4*. This can be done with Visual Paradigm. The complete Java code of the interfaces and a skeleton of the implementation class are shown below generated by Visual Paradigm is provided below:

```
public interface IAccountManagement {

    /**
     *
     * @param merchantID
     */
    Merchant getMerchant(String merchantID);

    /**
     *
     * @param merchant
     */
    boolean CreateMerchant(Merchant merchant);

    /**
     *
     * @param merchantID
     * @param newLimit
     */
    boolean UpdateCreditLimit(String merchantID, double newLimit);

    /**
     *
     * @param merchantID
     */
    double getMerchantBalance(String merchantID);
```

```java
    /**
     *
     * @param merchantID
     * @param payment
     */
    boolean ApplyPayment(String merchantID, Payment payment);

}
```

```java
import java.util.Date;

public interface IOrderManagement {

    /**
     * @param order
     */
    String createOrder(Order order);

    /**
     * @param orderID
     */
    Order getOrder(String orderID);

    /**
     * @param orderID
     * @param status
     */
    boolean updateOrderStatus(String orderID, String status);

    /**
     * @param merchantID
     */
    Order[] orderHistory(String merchantID);

    /**
     * @param orderID
     */
    double calculateOrderTotal(String orderID);

    /**
     * @param orderID
     */
    boolean cancelOrder(String orderID);

    /**
     * @param orderID
     */
    Invoice raiseInvoice(String orderID);

    boolean updateDispatchInfo(String orderID, String courier,
                               Date dispatchDate, String trackingNo);

    /**
     * @param merchantID
     * @param payment
```

```
     */
    boolean recordPayment(String merchantID, Payment payment);

    /**
     *
     * @param orderID
     * @param courier
     * @param dispatchDate
     * @param trackingNo
     */
}
```

```
public interface IProductCatalogue {

    /**
     *
     * @param productID
     */
    Product getProduct(String productID);

    Product[] listProducts();

    /**
     *
     * @param product
     */
    boolean addProduct(Product product);

    /**
     *
     * @param productID
     * @param quantity
     */
    boolean updateStock(String productID, int quantity);

    Product[] getLowStockProducts();

    /**
     *
     * @param productID
     * @param level
     */
    boolean setMinimumStockLevel(String productID, int level);

    /**
     *
     * @param itemID
     * @param quantity
     */
    void addStock(int itemID, int quantity);

}
```

```
public interface IReporting {
```

```
    /**
     *
     * @param merchantID
     */
    Report generateMerchantReport(String merchantID);

    Report generateStockReport();

    /**
     *
     * @param merchantID
     */
    Report generateOrderSummary(String merchantID);

    /**
     *
     * @param merchantID
     */
    Report generateSalesPerformanceReport(String merchantID);

}
```

```
public interface I_IPOS_CA {

    /**
     *
     * @param productID
     * @param stockAmount
     */
    int mutateStock(int productID, int stockAmount);

    /**
     *
     * @param productID
     */
    int StockQuantity(int productID);

}
```

```
// public interface I_IPOS_PU {
    /**
     *
     * @param sender
     * @param receivers
     * @param subject
     * @param body
     */
    //void sendEmail(string sender, string[] receivers, string subject, string
body);

    /**
     *
     * @param currency
     * @param amount
     * @param cardDetails
```

```
     */
    //int processPayment(char currency, float amount, Card cardDetails);

    /**
     *
     * @param orderID
     * @param newStatus
     */
    //void updateOrderStatus(int orderID, string newStatus);

// }
```

```
// AccountManagement_API.java
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class AccountManagement_API implements IAccountManagement {

    private Map<String, Merchant> merchants = new ConcurrentHashMap<>();
    private Map<String, Admin> admins = new ConcurrentHashMap<>();
    private Map<String, Manager> managers = new ConcurrentHashMap<>();
    private Map<String, DiscountPlan> discountPlans = new
ConcurrentHashMap<>();

    public AccountManagement_API() {
        // Initialize with some sample data
        initializeSampleData();
    }

    private void initializeSampleData() {
        // Create sample discount plans
        DiscountPlan fixedPlan = new DiscountPlan();
        fixedPlan.setPlanID("FIXED001");
        fixedPlan.setDescription("Fixed 5% discount on all orders");
        fixedPlan.setDiscountRate(5.0);
        discountPlans.put(fixedPlan.getPlanID(), fixedPlan);

        DiscountPlan flexiblePlan = new DiscountPlan();
        flexiblePlan.setPlanID("FLEX001");
        flexiblePlan.setDescription("Flexible discount: 1% (<1000), 2%
(1000-2000), 3% (>2000)");
        flexiblePlan.setDiscountRate(0.0); // Rate calculated based on order
value
        discountPlans.put(flexiblePlan.getPlanID(), flexiblePlan);

        // Create sample admin
        Admin admin = new Admin();
        admin.setAdminID("ADMIN001");
        admin.setName("System Admin");
        admin.setEmail("admin@infopharma.com");
        admin.setRole("Administrator");
        admins.put(admin.getAdminID(), admin);
    }
```

```java
    @Override
    public Merchant getMerchant(String merchantID) {
        if (merchantID == null || merchantID.trim().isEmpty()) {
            throw new IllegalArgumentException("Merchant ID cannot be null or
empty");
        }
        return merchants.get(merchantID);
    }

    @Override
    public boolean CreateMerchant(Merchant merchant) {
        if (merchant == null) {
            return false;
        }

        // Validate required fields
        if (merchant.getMerchantID() == null || merchant.getName() == null ||
                merchant.getAddress() == null) {
            return false;
        }

        // Check if merchant already exists
        if (merchants.containsKey(merchant.getMerchantID())) {
            return false;
        }

        // Set default values if not provided
        if (merchant.getCreditLimit() <= 0) {
            merchant.setCreditLimit(5000.0); // Default credit limit
        }

        if (merchant.getBalance() < 0) {
            merchant.setBalance(0.0);
        }

        if (merchant.getStatus() == null) {
            merchant.setStatus("ACTIVE");
        }

        merchants.put(merchant.getMerchantID(), merchant);
        return true;
    }

    @Override
    public boolean UpdateCreditLimit(String merchantID, double newLimit) {
        if (merchantID == null || newLimit < 0) {
            return false;
        }

        Merchant merchant = merchants.get(merchantID);
        if (merchant == null) {
            return false;
        }

        merchant.setCreditLimit(newLimit);
```

```
        return true;
    }

    @Override
    public double getMerchantBalance(String merchantID) {
        if (merchantID == null) {
            throw new IllegalArgumentException("Merchant ID cannot be null");
        }

        Merchant merchant = merchants.get(merchantID);
        if (merchant == null) {
            throw new IllegalArgumentException("Merchant not found: " +
merchantID);
        }

        return merchant.getBalance();
    }

    @Override
    public boolean ApplyPayment(String merchantID, Payment payment) {
        if (merchantID == null || payment == null) {
            return false;
        }

        Merchant merchant = merchants.get(merchantID);
        if (merchant == null) {
            return false;
        }

        // Update merchant balance (reduce debt)
        double currentBalance = merchant.getBalance();
        merchant.setBalance(currentBalance - payment.getAmount());

        // Update merchant status based on payment
        updateMerchantStatusAfterPayment(merchant);

        return true;
    }

    private void updateMerchantStatusAfterPayment(Merchant merchant) {
        double balance = merchant.getBalance();
        String currentStatus = merchant.getStatus();

        if (balance <= 0) {
            merchant.setStatus("ACTIVE");
        } else if (balance > 0 && balance <= merchant.getCreditLimit()) {
            if ("SUSPENDED".equals(currentStatus) ||
"IN_DEFAULT".equals(currentStatus)) {
                // Keep as is, manager must reactivate
            } else {
                merchant.setStatus("ACTIVE");
            }
        }
    }
```

```java
    public boolean createAdminAccount(Admin admin) {
        if (admin == null || admin.getAdminID() == null || admin.getEmail() ==
null) {
            return false;
        }

        if (admins.containsKey(admin.getAdminID())) {
            return false;
        }

        if (admin.getRole() == null) {
            admin.setRole("Administrator");
        }

        admins.put(admin.getAdminID(), admin);
        return true;
    }

    public boolean createManagerAccount(Manager manager) {
        if (manager == null || manager.getManagerID() == null ||
manager.getEmail() == null) {
            return false;
        }

        if (managers.containsKey(manager.getManagerID())) {
            return false;
        }

        managers.put(manager.getManagerID(), manager);
        return true;
    }

    public boolean updateDiscountPlan(String merchantID, DiscountPlan plan) {
        if (merchantID == null || plan == null) {
            return false;
        }

        Merchant merchant = merchants.get(merchantID);
        if (merchant == null) {
            return false;
        }

        // Store discount plan association
        discountPlans.put(merchantID + "_PLAN", plan);
        return true;
    }

    public boolean changeAccountStatus(String accountID, String status) {
        if (accountID == null || status == null) {
            return false;
        }

        // Check if it's a merchant account
        Merchant merchant = merchants.get(accountID);
        if (merchant != null) {
```

```
        // Only manager can change status from IN_DEFAULT
        if ("IN_DEFAULT".equals(merchant.getStatus()) &&
              !"ACTIVE".equals(status) && !"SUSPENDED".equals(status)) {
           return false;
        }
        merchant.setStatus(status);
        return true;
     }

     // Check if it's an admin account
     Admin admin = admins.get(accountID);
     if (admin != null) {
        admin.setRole(status);
        return true;
     }

     // Check if it's a manager account
     Manager manager = managers.get(accountID);
     if (manager != null) {
        manager.setRegion(status);
        return true;
     }

     return false;
  }

  public DiscountPlan getDiscountPlanForMerchant(String merchantID) {
     return discountPlans.get(merchantID + "_PLAN");
  }

  public List<Merchant> getAllMerchants() {
     return new ArrayList<>(merchants.values());
  }

  public List<Admin> getAllAdmins() {
     return new ArrayList<>(admins.values());
  }

  public List<Manager> getAllManagers() {
     return new ArrayList<>(managers.values());
  }
}
```

```
// OrderManagment_API.java (fixed class name typo)
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.Date;

public class OrderManagment_API implements IOrderManagement {

   private Map<String, Order> orders = new ConcurrentHashMap<>();
   private Map<String, Invoice> invoices = new ConcurrentHashMap<>();
   private Map<String, List<Order>> merchantOrders = new
ConcurrentHashMap<>();
```

```
   private Map<String, List<Payment>> merchantPayments = new
ConcurrentHashMap<>();
   private AccountManagement_API accountManagement;
   private ProductCatalogue_API productCatalogue;
   private AtomicInteger orderCounter = new AtomicInteger(1000);
   private AtomicInteger invoiceCounter = new AtomicInteger(5000);

   public OrderManagment_API(AccountManagement_API accountManagement,
                             ProductCatalogue_API productCatalogue) {
      this.accountManagement = accountManagement;
      this.productCatalogue = productCatalogue;
   }

   @Override
   public String createOrder(Order order) {
      if (order == null) {
         throw new IllegalArgumentException("Order cannot be null");
      }

      // Validate merchant
      Merchant merchant =
accountManagement.getMerchant(order.getMerchantID());
      if (merchant == null) {
         throw new IllegalArgumentException("Merchant not found: " +
order.getMerchantID());
      }

      // Check if merchant account is active
      if (!"ACTIVE".equals(merchant.getStatus())) {
         throw new IllegalStateException("Merchant account is not active.
Status: " +
               merchant.getStatus());
      }

      // Check credit limit
      double orderTotal = calculateOrderTotalFromItems(order.getItems());
      if (merchant.getBalance() + orderTotal > merchant.getCreditLimit()) {
         throw new IllegalStateException("Order would exceed credit limit. "
+
               "Current balance: " + merchant.getBalance() +
               ", Order total: " + orderTotal +
               ", Credit limit: " + merchant.getCreditLimit());
      }

      // Generate order ID
      String orderID = "ORD" + orderCounter.incrementAndGet();
      order.setOrderID(orderID);

      if (order.getOrderDate() == null) {
         order.setOrderDate(new Date());
      }

      if (order.getStatus() == null) {
         order.setStatus("PENDING");
      }
```

```java
        // Store order
        orders.put(orderID, order);

        // Add to merchant's order history
        merchantOrders.computeIfAbsent(order.getMerchantID(),
                k -> new ArrayList<>()).add(order);

        // Update stock levels (reduce by ordered quantities)
        updateStockForOrder(order.getItems());

        // Update merchant balance (increase debt)
        merchant.setBalance(merchant.getBalance() + orderTotal);

        return orderID;
    }

    private double calculateOrderTotalFromItems(OrderItem[] items) {
        if (items == null) return 0.0;

        double total = 0.0;
        for (OrderItem item : items) {
            total += item.getQuantity() * item.getUnitPrice();
        }
        return total;
    }

    private void updateStockForOrder(OrderItem[] items) {
        if (items == null) return;

        for (OrderItem item : items) {
            Product product = productCatalogue.getProduct(item.getProductID());
            if (product != null) {
                int newStock = product.getStockLevel() - item.getQuantity();
                product.setStockLevel(Math.max(0, newStock));
                productCatalogue.updateStock(item.getProductID(), newStock);
            }
        }
    }

    @Override
    public Order getOrder(String orderID) {
        if (orderID == null) {
            throw new IllegalArgumentException("Order ID cannot be null");
        }
        return orders.get(orderID);
    }

    @Override
    public boolean updateOrderStatus(String orderID, String status) {
        if (orderID == null || status == null) {
            return false;
        }

        Order order = orders.get(orderID);
```

```java
        if (order == null) {
            return false;
        }

        order.setStatus(status);
        return true;
    }

    @Override
    public Order[] orderHistory(String merchantID) {
        if (merchantID == null) {
            return new Order[0];
        }

        List<Order> merchantOrderList = merchantOrders.get(merchantID);
        if (merchantOrderList == null) {
            return new Order[0];
        }

        return merchantOrderList.toArray(new Order[0]);
    }

    @Override
    public double calculateOrderTotal(String orderID) {
        if (orderID == null) {
            return 0.0;
        }

        Order order = orders.get(orderID);
        if (order == null) {
            return 0.0;
        }

        return calculateOrderTotalFromItems(order.getItems());
    }

    @Override
    public boolean cancelOrder(String orderID) {
        if (orderID == null) {
            return false;
        }

        Order order = orders.get(orderID);
        if (order == null) {
            return false;
        }

        // Can only cancel pending orders
        if (!"PENDING".equals(order.getStatus())) {
            return false;
        }

        order.setStatus("CANCELLED");

        // Restore stock
```

```
        restoreStockForCancelledOrder(order.getItems());

        // Update merchant balance
        Merchant merchant =
accountManagement.getMerchant(order.getMerchantID());
        if (merchant != null) {
            double orderTotal = calculateOrderTotalFromItems(order.getItems());
            merchant.setBalance(merchant.getBalance() - orderTotal);
        }

        return true;
    }

    private void restoreStockForCancelledOrder(OrderItem[] items) {
        if (items == null) return;

        for (OrderItem item : items) {
            Product product = productCatalogue.getProduct(item.getProductID());
            if (product != null) {
                int newStock = product.getStockLevel() + item.getQuantity();
                product.setStockLevel(newStock);
                productCatalogue.updateStock(item.getProductID(), newStock);
            }
        }
    }

    @Override
    public Invoice raiseInvoice(String orderID) {
        if (orderID == null) {
            throw new IllegalArgumentException("Order ID cannot be null");
        }

        Order order = orders.get(orderID);
        if (order == null) {
            throw new IllegalArgumentException("Order not found: " + orderID);
        }

        // Check if invoice already exists
        for (Invoice inv : invoices.values()) {
            if (orderID.equals(inv.getOrderID())) {
                return inv; // Return existing invoice
            }
        }

        // Create new invoice
        Invoice invoice = new Invoice();
        String invoiceID = "INV" + invoiceCounter.incrementAndGet();
        invoice.setInvoiceID(invoiceID);
        invoice.setOrderID(orderID);
        invoice.setMerchantID(order.getMerchantID());
        invoice.setIssueDate(new Date());

        // Calculate due date (30 days from issue)
        Calendar cal = Calendar.getInstance();
        cal.add(Calendar.DAY_OF_MONTH, 30);
```

```java
        invoice.setDueDate(cal.getTime());

        invoice.setTotalAmount(calculateOrderTotal(orderID));
        invoice.setStatus("ISSUED");

        invoices.put(invoiceID, invoice);

        return invoice;
    }

    @Override
    public boolean updateDispatchInfo(String orderID, String courier, Date
dispatchDate, String trackingNo) {
        return false;
    }


    @Override
    public boolean recordPayment(String merchantID, Payment payment) {
        if (merchantID == null || payment == null) {
            return false;
        }

        // Store payment
        merchantPayments.computeIfAbsent(merchantID, k -> new
ArrayList<>()).add(payment);

        // Apply payment to merchant account
        return accountManagement.ApplyPayment(merchantID, payment);
    }

    public List<Invoice> getInvoicesForMerchant(String merchantID) {
        List<Invoice> result = new ArrayList<>();
        for (Invoice invoice : invoices.values()) {
            if (merchantID.equals(invoice.getMerchantID())) {
                result.add(invoice);
            }
        }
        return result;
    }

    public List<Payment> getPaymentsForMerchant(String merchantID) {
        return merchantPayments.getOrDefault(merchantID, new ArrayList<>());
    }

    public Invoice getInvoice(String invoiceID) {
        return invoices.get(invoiceID);
    }
}
```

```java
// ProductCatalogue_API.java
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.stream.Collectors;
```

```java
public class ProductCatalogue_API implements IProductCatalogue {

   private Map<String, Product> products = new ConcurrentHashMap<>();

   public ProductCatalogue_API() {
      initializeSampleCatalogue();
   }

   private void initializeSampleCatalogue() {
      // Add sample products based on Appendix 1
      addSampleProduct("10000001", "Paracetamol", 0.10, 10345, 300);
      addSampleProduct("10000002", "Aspirin", 0.50, 12453, 500);
      addSampleProduct("10000003", "Analgin", 1.20, 4235, 200);
      addSampleProduct("10000004", "Celebrex, caps 100 mg", 10.00, 3420,
200);
      addSampleProduct("10000005", "Celebrex, caps 200 mg", 18.50, 1450,
150);
      addSampleProduct("10000006", "Retin-A Tretin, 30 g", 25.00, 2013, 200);
      addSampleProduct("10000007", "Lipitor TB, 20 mg", 15.50, 1562, 200);
      addSampleProduct("10000008", "Claritin CR, 60g", 19.50, 2540, 200);
      addSampleProduct("20000004", "Iodine tincture", 0.30, 2213, 200);
      addSampleProduct("20000005", "Rhynol", 2.50, 1908, 300);
      addSampleProduct("30000001", "Ospen", 10.50, 809, 200);
      addSampleProduct("30000002", "Ampolen", 15.00, 1340, 300);
      addSampleProduct("40000001", "Vitamin C", 1.20, 3258, 300);
      addSampleProduct("40000002", "Vitamin B12", 1.30, 2673, 300);
   }

   private void addSampleProduct(String id, String name, double price,
                                 int stock, int minStock) {
      Product product = new Product();
      product.setProductID(id);
      product.setName(name);
      product.setPrice(price);
      product.setStockLevel(stock);
      product.setMinimumStockLevel(minStock);
      products.put(id, product);
   }

   @Override
   public Product getProduct(String productID) {
      if (productID == null) {
         throw new IllegalArgumentException("Product ID cannot be null");
      }
      return products.get(productID);
   }

   @Override
   public Product[] listProducts() {
      return products.values().toArray(new Product[0]);
   }

   @Override
   public boolean addProduct(Product product) {
      if (product == null || product.getProductID() == null) {
```

```java
        return false;
    }

    if (products.containsKey(product.getProductID())) {
        return false;
    }

    // Set default minimum stock if not set
    if (product.getMinimumStockLevel() <= 0) {
        product.setMinimumStockLevel(10);
    }

    products.put(product.getProductID(), product);
    return true;
}

@Override
public boolean updateStock(String productID, int quantity) {
    if (productID == null || quantity < 0) {
        return false;
    }

    Product product = products.get(productID);
    if (product == null) {
        return false;
    }

    product.setStockLevel(quantity);
    return true;
}

@Override
public Product[] getLowStockProducts() {
    List<Product> lowStock = products.values().stream()
            .filter(p -> p.getStockLevel() < p.getMinimumStockLevel())
            .collect(Collectors.toList());

    return lowStock.toArray(new Product[0]);
}

@Override
public boolean setMinimumStockLevel(String productID, int level) {
    if (productID == null || level < 0) {
        return false;
    }

    Product product = products.get(productID);
    if (product == null) {
        return false;
    }

    product.setMinimumStockLevel(level);
    return true;
}
```

```java
    @Override
    public void addStock(int itemID, int quantity) {
        // Convert int ID to String ID
        String productID = String.valueOf(itemID);
        // Pad with zeros to match format (e.g., 10000001)
        while (productID.length() < 8) {
            productID = "0" + productID;
        }

        Product product = products.get(productID);
        if (product != null && quantity > 0) {
            int newStock = product.getStockLevel() + quantity;
            product.setStockLevel(newStock);
        }
    }

    // Additional method for adding stock with String ID
    public void addStock(String productID, int quantity) {
        if (productID == null || quantity <= 0) return;

        Product product = products.get(productID);
        if (product != null) {
            int newStock = product.getStockLevel() + quantity;
            product.setStockLevel(newStock);
        }
    }

    public List<Product> searchProducts(String keyword) {
        if (keyword == null || keyword.trim().isEmpty()) {
            return new ArrayList<>(products.values());
        }

        String lowerKeyword = keyword.toLowerCase();
        return products.values().stream()
                .filter(p -> p.getName().toLowerCase().contains(lowerKeyword) ||
                        p.getProductID().contains(keyword))
                .collect(Collectors.toList());
    }
}
```

```java
// Reporting_API.java
import java.text.SimpleDateFormat;
import java.util.*;
import java.util.stream.Collectors;

public class Reporting_API implements IReporting {

    private AccountManagement_API accountManagement;
    private OrderManagment_API orderManagement;
    private ProductCatalogue_API productCatalogue;

    public Reporting_API(AccountManagement_API accountManagement,
                         OrderManagment_API orderManagement,
                         ProductCatalogue_API productCatalogue) {
        this.accountManagement = accountManagement;
```

```
        this.orderManagement = orderManagement;
        this.productCatalogue = productCatalogue;
    }

    @Override
    public Report generateMerchantReport(String merchantID) {
        if (merchantID == null) {
            throw new IllegalArgumentException("Merchant ID cannot be null");
        }

        Merchant merchant = accountManagement.getMerchant(merchantID);
        if (merchant == null) {
            throw new IllegalArgumentException("Merchant not found: " +
merchantID);
        }

        StringBuilder content = new StringBuilder();
        content.append("MERCHANT REPORT\n");
        content.append("================\n\n");
        content.append("Merchant ID:
").append(merchant.getMerchantID()).append("\n");
        content.append("Name: ").append(merchant.getName()).append("\n");
        content.append("Address: ").append(merchant.getAddress()).append("\n");
        content.append("Credit Limit: £").append(String.format("%.2f",
merchant.getCreditLimit())).append("\n");
        content.append("Current Balance: £").append(String.format("%.2f",
merchant.getBalance())).append("\n");
        content.append("Status: ").append(merchant.getStatus()).append("\n\n");

        // Add order history
        content.append("ORDER HISTORY:\n");
        content.append("--------------\n");
        Order[] orders = orderManagement.orderHistory(merchantID);
        if (orders.length == 0) {
            content.append("No orders found.\n");
        } else {
            double totalOrderValue = 0;
            for (Order order : orders) {
                content.append("Order ID:
").append(order.getOrderID()).append("\n");
                content.append("  Date:
").append(order.getOrderDate()).append("\n");
                content.append("  Status:
").append(order.getStatus()).append("\n");
                double orderTotal =
orderManagement.calculateOrderTotal(order.getOrderID());
                content.append("  Total: £").append(String.format("%.2f",
orderTotal)).append("\n");
                totalOrderValue += orderTotal;
            }
            content.append("\nTotal Value of All Orders:
£").append(String.format("%.2f", totalOrderValue)).append("\n");
        }

        // Add payment history
```

```
        content.append("\nPAYMENT HISTORY:\n");
        content.append("----------------\n");
        List<Payment> payments =
orderManagement.getPaymentsForMerchant(merchantID);
        if (payments.isEmpty()) {
            content.append("No payments recorded.\n");
        } else {
            double totalPayments = 0;
            for (Payment payment : payments) {
                content.append("Payment ID:
").append(payment.getPaymentID()).append("\n");
                content.append("  Date:
").append(payment.getDate()).append("\n");
                content.append("  Amount: £").append(String.format("%.2f",
payment.getAmount())).append("\n");
                totalPayments += payment.getAmount();
            }
            content.append("\nTotal Payments: £").append(String.format("%.2f",
totalPayments)).append("\n");
        }

        Report report = new Report();
        report.setTitle("Merchant Report - " + merchant.getName());
        report.setGeneratedOn(new Date());
        report.setContent(content.toString());

        return report;
    }

    @Override
    public Report generateStockReport() {
        StringBuilder content = new StringBuilder();
        content.append("STOCK REPORT\n");
        content.append("============\n\n");

        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        content.append("Generated: ").append(dateFormat.format(new
Date())).append("\n\n");

        Product[] products = productCatalogue.listProducts();

        // Sort by product ID
        Arrays.sort(products, (p1, p2) ->
p1.getProductID().compareTo(p2.getProductID()));

        content.append(String.format("%-10s %-30s %-10s %-15s %-15s %-15s\n",
                "Item ID", "Description", "Price", "Stock Level", "Min Stock",
"Status"));
        content.append(String.format("%-10s %-30s %-10s %-15s %-15s %-15s\n",
                "-------", "-----------", "-----", "-----------", "---------",
"------"));

        int totalItems = 0;
        int lowStockCount = 0;
        double totalStockValue = 0;
```

```
        for (Product product : products) {
            String status = product.getStockLevel() <
product.getMinimumStockLevel() ?
                    "LOW STOCK" : "OK";
            if ("LOW STOCK".equals(status)) lowStockCount++;

            content.append(String.format("%-10s %-30s £%-9.2f %-15d %-15d
%-15s\n",
                    product.getProductID(),
                    truncate(product.getName(), 28),
                    product.getPrice(),
                    product.getStockLevel(),
                    product.getMinimumStockLevel(),
                    status));

            totalItems++;
            totalStockValue += product.getPrice() * product.getStockLevel();
        }

        content.append("\n");
        content.append("SUMMARY:\n");
        content.append("--------\n");
        content.append("Total Products: ").append(totalItems).append("\n");
        content.append("Low Stock Items: ").append(lowStockCount).append("\n");
        content.append("Total Stock Value: £").append(String.format("%.2f",
totalStockValue)).append("\n");

        // Add recommended orders for low stock items
        if (lowStockCount > 0) {
            content.append("\nRECOMMENDED ORDERS (Low Stock Items):\n");
            content.append("-------------------------------------\n");
            content.append(String.format("%-10s %-30s %-15s %-20s\n",
                    "Item ID", "Description", "Current Stock", "Recommended
Order"));

            for (Product product : products) {
                if (product.getStockLevel() < product.getMinimumStockLevel()) {
                    int recommendedOrder = (int)(product.getMinimumStockLevel() *
1.1) - product.getStockLevel();
                    content.append(String.format("%-10s %-30s %-15d %-20d\n",
                            product.getProductID(),
                            truncate(product.getName(), 28),
                            product.getStockLevel(),
                            Math.max(0, recommendedOrder)));
                }
            }
        }

        Report report = new Report();
        report.setTitle("Stock Report");
        report.setGeneratedOn(new Date());
        report.setContent(content.toString());

        return report;
```

```
    }

    @Override
    public Report generateOrderSummary(String merchantID) {
        if (merchantID == null) {
            throw new IllegalArgumentException("Merchant ID cannot be null");
        }

        Merchant merchant = accountManagement.getMerchant(merchantID);
        if (merchant == null) {
            throw new IllegalArgumentException("Merchant not found: " +
merchantID);
        }

        StringBuilder content = new StringBuilder();
        content.append("ORDER SUMMARY\n");
        content.append("=============\n\n");

        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        content.append("Merchant: ").append(merchant.getName()).append("\n");
        content.append("Account:
").append(merchant.getMerchantID()).append("\n");
        content.append("Generated: ").append(dateFormat.format(new
Date())).append("\n\n");

        content.append("CONTACT DETAILS:\n");
        content.append("----------------\n");
        content.append(merchant.getAddress()).append("\n\n");

        content.append("ORDER HISTORY SUMMARY:\n");
        content.append("----------------------\n");

        Order[] orders = orderManagement.orderHistory(merchantID);

        // Filter orders for current month
        Calendar cal = Calendar.getInstance();
        int currentMonth = cal.get(Calendar.MONTH);
        int currentYear = cal.get(Calendar.YEAR);

        List<Order> currentMonthOrders = Arrays.stream(orders)
                .filter(o -> {
                    cal.setTime(o.getOrderDate());
                    return cal.get(Calendar.MONTH) == currentMonth &&
                            cal.get(Calendar.YEAR) == currentYear;
                })
                .collect(Collectors.toList());

        content.append(String.format("%-10s %-12s %-12s %-12s %-15s\n",
                "Order ID", "Date", "Amount", "Status", "Payment Status"));

        double totalAmount = 0;
        int pendingCount = 0;

        for (Order order : currentMonthOrders) {
```

```
        double amount =
orderManagement.calculateOrderTotal(order.getOrderID());
        String paymentStatus = getPaymentStatus(order, merchant);

        content.append(String.format("%-10s %-12s £%-11.2f %-12s %-15s\n",
              order.getOrderID(),
              dateFormat.format(order.getOrderDate()),
              amount,
              order.getStatus(),
              paymentStatus));

        totalAmount += amount;
        if ("PENDING".equals(paymentStatus)) pendingCount++;
      }

      content.append("\n");
      content.append(String.format("%-10s %-12s £%-11.2f\n",
            "TOTAL:", "", totalAmount));
      content.append("Pending Payments: ").append(pendingCount).append("\n");

      Report report = new Report();
      report.setTitle("Order Summary - " + merchant.getName());
      report.setGeneratedOn(new Date());
      report.setContent(content.toString());

      return report;
   }

   private String getPaymentStatus(Order order, Merchant merchant) {
      // Simplified payment status logic
      double orderTotal =
orderManagement.calculateOrderTotal(order.getOrderID());
      if ("DISPATCHED".equals(order.getStatus()) ||
"DELIVERED".equals(order.getStatus())) {
         // Check if paid based on merchant balance
         if (merchant.getBalance() <= 0) {
            return "PAID";
         } else {
            return "PENDING";
         }
      }
      return "NOT DUE";
   }

   @Override
   public Report generateSalesPerformanceReport(String merchantID) {
      if (merchantID == null) {
         throw new IllegalArgumentException("Merchant ID cannot be null");
      }

      Merchant merchant = accountManagement.getMerchant(merchantID);
      if (merchant == null) {
         throw new IllegalArgumentException("Merchant not found: " +
merchantID);
      }
```

```
        StringBuilder content = new StringBuilder();
        content.append("SALES PERFORMANCE REPORT\n");
        content.append("=======================\n\n");

        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        content.append("Merchant: ").append(merchant.getName()).append("\n");
        content.append("Generated: ").append(dateFormat.format(new
Date())).append("\n\n");

        Order[] orders = orderManagement.orderHistory(merchantID);

        // Group orders by month
        Map<String, List<Order>> ordersByMonth = new HashMap<>();
        SimpleDateFormat monthFormat = new SimpleDateFormat("MMMM yyyy");

        for (Order order : orders) {
            String month = monthFormat.format(order.getOrderDate());
            ordersByMonth.computeIfAbsent(month, k -> new
ArrayList<>()).add(order);
        }

        content.append(String.format("%-15s %-15s %-15s %-15s\n",
                "Month", "Orders", "Total Sales", "Avg Order Value"));

        List<String> months = new ArrayList<>(ordersByMonth.keySet());
        Collections.sort(months);

        double totalSales = 0;
        int totalOrders = 0;

        for (String month : months) {
            List<Order> monthOrders = ordersByMonth.get(month);
            double monthTotal = 0;
            for (Order order : monthOrders) {
                monthTotal +=
orderManagement.calculateOrderTotal(order.getOrderID());
            }
            double avgOrder = monthOrders.isEmpty() ? 0 : monthTotal /
monthOrders.size();

            content.append(String.format("%-15s %-15d £%-14.2f £%-14.2f\n",
                    month, monthOrders.size(), monthTotal, avgOrder));

            totalSales += monthTotal;
            totalOrders += monthOrders.size();
        }

        content.append("\n");
        content.append("TOTALS:\n");
        content.append("-------\n");
        content.append("Total Orders: ").append(totalOrders).append("\n");
        content.append("Total Sales: £").append(String.format("%.2f",
totalSales)).append("\n");
        content.append("Average Order Value: £").append(
```

```
            totalOrders == 0 ? "0.00" : String.format("%.2f", totalSales /
totalOrders)).append("\n");

      Report report = new Report();
      report.setTitle("Sales Performance - " + merchant.getName());
      report.setGeneratedOn(new Date());
      report.setContent(content.toString());

      return report;
   }

   // Additional reports required for IPOS-SA
   public Report generateOverduePaymentsReport() {
      StringBuilder content = new StringBuilder();
      content.append("OVERDUE PAYMENTS REPORT\n");
      content.append("=======================\n\n");

      SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
      content.append("Generated: ").append(dateFormat.format(new
Date())).append("\n\n");

      Calendar cal = Calendar.getInstance();
      cal.add(Calendar.DAY_OF_MONTH, -30); // 30 days overdue threshold

      List<Merchant> allMerchants = accountManagement.getAllMerchants();
      List<Merchant> overdueMerchants = new ArrayList<>();

      for (Merchant merchant : allMerchants) {
         if (merchant.getBalance() > 0) {
            // Check if any invoices are overdue
            List<Invoice> invoices = orderManagement.getInvoicesForMerchant(
                  merchant.getMerchantID());
            for (Invoice invoice : invoices) {
               if (invoice.getDueDate().before(new Date()) &&
                     !"PAID".equals(invoice.getStatus())) {
                  overdueMerchants.add(merchant);
                  break;
               }
            }
         }
      }

      if (overdueMerchants.isEmpty()) {
         content.append("No overdue payments found.\n");
      } else {
         content.append(String.format("%-15s %-30s %-15s %-15s\n",
               "Merchant ID", "Name", "Balance", "Status"));

         for (Merchant merchant : overdueMerchants) {
            content.append(String.format("%-15s %-30s £%-14.2f %-15s\n",
                  merchant.getMerchantID(),
                  truncate(merchant.getName(), 28),
                  merchant.getBalance(),
                  merchant.getStatus()));
         }
```

```
        }

        Report report = new Report();
        report.setTitle("Overdue Payments Report");
        report.setGeneratedOn(new Date());
        report.setContent(content.toString());

        return report;
    }

    public Report generateTurnoverReport(DateRange period) {
        StringBuilder content = new StringBuilder();
        content.append("TURNOVER REPORT\n");
        content.append("===============\n\n");

        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        content.append("Period: ").append(dateFormat.format(period.getStart()))
              .append(" to
").append(dateFormat.format(period.getEnd())).append("\n\n");

        // Get all orders within period
        List<Order> ordersInPeriod = new ArrayList<>();
        Product[] allProducts = productCatalogue.listProducts();

        // Calculate sales
        Map<String, Integer> quantitySold = new HashMap<>();
        Map<String, Double> revenueByProduct = new HashMap<>();
        double totalRevenue = 0;
        int totalItemsSold = 0;

        for (Merchant merchant : accountManagement.getAllMerchants()) {
            Order[] orders =
orderManagement.orderHistory(merchant.getMerchantID());
            for (Order order : orders) {
                if (order.getOrderDate().after(period.getStart()) &&
                        order.getOrderDate().before(period.getEnd()) &&
                        !"CANCELLED".equals(order.getStatus())) {

                    ordersInPeriod.add(order);

                    // Calculate items sold
                    for (OrderItem item : order.getItems()) {
                        String productID = item.getProductID();
                        int qty = item.getQuantity();
                        double itemRevenue = qty * item.getUnitPrice();

                        quantitySold.put(productID,
                              quantitySold.getOrDefault(productID, 0) + qty);
                        revenueByProduct.put(productID,
                              revenueByProduct.getOrDefault(productID, 0.0) +
itemRevenue);

                        totalItemsSold += qty;
                        totalRevenue += itemRevenue;
                    }
```

```
            }
         }
      }

      content.append(String.format("%-10s %-30s %-15s %-15s\n",
            "Item ID", "Description", "Quantity Sold", "Revenue"));

      for (Product product : allProducts) {
         int qty = quantitySold.getOrDefault(product.getProductID(), 0);
         double revenue =
revenueByProduct.getOrDefault(product.getProductID(), 0.0);

         if (qty > 0) {
            content.append(String.format("%-10s %-30s %-15d £%-14.2f\n",
                  product.getProductID(),
                  truncate(product.getName(), 28),
                  qty,
                  revenue));
         }
      }

      content.append("\n");
      content.append("SUMMARY:\n");
      content.append("--------\n");
      content.append("Total Orders:
").append(ordersInPeriod.size()).append("\n");
      content.append("Total Items Sold:
").append(totalItemsSold).append("\n");
      content.append("Total Revenue: £").append(String.format("%.2f",
totalRevenue)).append("\n");

      Report report = new Report();
      report.setTitle("Turnover Report");
      report.setGeneratedOn(new Date());
      report.setContent(content.toString());

      return report;
   }

   private String truncate(String str, int maxLength) {
      if (str == null) return "";
      if (str.length() <= maxLength) return str;
      return str.substring(0, maxLength - 3) + "...";
   }
}
```

From these snippets of Java code, one can generate jUnit tests, if necessary for each of them, and **develop test cases** before the code of the implementation class(es) is available.

It is also useful at this stage to refine the generated code by **extending the comments** generated by Visual Paradigm and provide short descriptions of:

- What is the **purpose of each of the methods** defined in the interfaces above?
- What **purpose does each of the parameters** of the interface methods serve? You may elaborate on any constraints that apply to the parameter values that you envisage. For instance, searching for a customer by name implies that a correctly implemented subsystem will find a matching record only if the customer is already in the respective

datastore (a file, a DB table, etc.) that the search is undertaken over. Similarly, the range of some of the parameters may be constrained, e.g. making an order if the account was downgraded by administrator if IPOS should result in order being declined, etc.

7.7 Testing the communications between subsystems

Now that we have defined the interfaces necessary for communication between the subsystems of IPOS, and generated Java code for all of them, we develop tests for inter-subsystem communication, i.e. of the respective implementation classes. Here we sketch tests for all methods of the implementation class IPOS_SA_API without committing to a particular testing platform to run the tests (e.g. jUnit).

Test number: ACC-01	
Objective: Test the account creation functionality by creating a merchant account (full details, credit limit > 0 FIXED discount)	
Set up: User is logged in as an administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System ensures there are no merchant accounts with identical details 3. A merchant account is created with the details input by the user 4. Account creation is confirmed	
Test: 1. User goes to the account creation page 2. User input full details into merchant details 3. User inputs credit limit > 0 4. User inputs FIXED discount 5. Account creation request is submitted	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-02	
Objective: Test the account creation functionality by not inputting a mandatory field (Missing contact details OR creditLimit)	
Set up: User is logged in as an administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies mandatory fields as missing 3. System notifies the user that there are missing mandatory fields 4. System does NOT create a new merchant account	
Test: 1. User goes to the account creation page 2. User inputs partial details into merchant details (not inputting contact details or creditLimit) 3. Account creation request is submitted	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-03	
Objective: Test the account creation functionality by inputting an invalid discount rate (FIXED rate < 0 or >100%	
Set up: User is logged in as an administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies discount rate as invalid 3. System notifies the user that the discount rate is invalid 4. System does NOT create a new merchant account	
Test: 1. User goes to the account creation page 2. User inputs full details into merchant details but with invalid discount rate 3. Account creation request is submitted	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-04	
Objective: Test the account creation functionality by inputting a boundary credit limit (creditLimit = 0)	
Set up: User is logged in as an administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System ensures there are no merchant accounts with identical details 3. A merchant account is created with the details input by the user 4. Account creation is confirmed	
Test: 1. User goes to the account creation page 2. User input full details into merchant details 3. User inputs credit limit = 0 4. Account creation request is submitted	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result: Passed	

Test number: ACC-05	
Objective: Test the account update functionality by updating a merchant's credit limit (newLimit > 0)	
Set up: User is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies the merchant with the given merchantId 3. System updates the new credit limit in the database 4. System confirms update with the user	
Test: 1. User navigates to the account update page 2. User identifies target merchant with merchantID 3. User choses to update credit limit and inputs new limit (newLimit > 0) 4. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-06	
Objective: Test the account update functionality by updating a merchant credit limit with invalid details (newLimit < 0)	
Set up: User is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies the credit limit as invalid 3. System informs the user that the given credit limit is invalid 4. System does NOT update merchant credit limit	
Test: 1. User navigates to the account update page 2. User identifies target merchant with merchantID 3. User choses to update credit limit and inputs new limit (newLimit < 0) 4. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-07	
Objective: Test the account update functionality by updating a merchant discount plan (switch to FLEXIBLE with valid thresholds)	
Set up: User is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies the merchant with the given merchantID 3. System updates the discount plan 4. System confirms update with the user	
Test: 1. User navigates to the account update page 2. User identifies target merchant with merchantID 3. User choses to update discount plan and inputs plan (FLEXIBLE with valid thresholds) 4. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-08	
Objective: Test the account update functionality by updating a merchant discount plan (discount plan = null)	
Set up: User is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives the input data 2. System identifies the discount plan as invalid 3. System informs the user that the given credit limit is invalid 4. System does NOT update merchant discount plan	
Test: 1. User navigates to the account update page 2. User identifies target merchant with merchantID 3. User choses to update discount plan and inputs no discount plan (discountPlan = null) 4. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-09	
Objective: Test changing merchant status by suspending merchant account	
Set up: User is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives request to update merchant status 2. System identifies merchant with provided merchantID 3. System updates merchant status in database (updates merchat status to SUSPENDED) 4. System notifies user of update	
Test: 1. User navigates to account update page 2. User identifies target merchant with merchantID 3. User choses to SUSPEND merchant account 4. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-10	
Objective: Prevent unauthorised users from changing the merchant status	
Set up: User is logged in as merchant, all systems working as intended	
Expected result: 1. System receives request to update merchant status 2. System recognises account as not having authorisation to update status 3. System notifies user of insufficient authorisation 4. System does NOT update merchant status	
Test: 1. User navigates to account update page 2. User attempts to update merchant status 3. User submits account update request	
Test record:	
Date: 18/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-11	
Objective: Ensure a user cannot restore IN_DEFAULT without director approval	
Set up: The user is logged in as a manager, all systems are working as intended	
Expected result: 1. System receives a request to update merchant status 2. System identifies merchant with provided merchantID 3. System checks to see if approval has been given 4. System inform the user of no director authorisation 5. System does NOT update merchant status	
Test: 1. User navigates to account update page 2. User identifies target merchant (must currently be suspended, with no auth to unsuspend) with merchantID 3. User selects to unsuspend account 4. User submits account update request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-12	
Objective: Test that the system can retrieve a merchant when requested	
Set up: The user is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives a request to retrieve merchant 2. System identifies target merchant with merchantID 3. System retrieves merchant	
Test: 1. User identifies target merchant with merchantID 2. User sends system request to retrieve merchant	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: ACC-13	
Objective: Test that the system can handle merchant retrieval request with an invalid merchantID	
Set up: The user is logged in as a manager or administrator, all systems working as intended	
Expected result: 1. System receives a request to retrieve a merchant 2. System is unable to find the target merchant with the given merchantID 3. System informs the user that the merchant cannot be found	
Test: 1. User prepares retrieve merchant request with invalid merchantID 2. User submits retrieve merchant request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-01	
Objective: Ensure the user can add an item to the database (unique itemID, price >=0, availability >=0)	
Set up: The user is logged in as an administrator, all systems are working as intended	
Expected result: 1. System receives a request to add a new item to the database 2. System confirms all input fields are valid 3. System adds item to the database 4. System confirms to the user that the item has been added	
Test: 1. User navigates to catalogue page 2. User selects to add a new item 3. User inputs item details (unique itemID, price >= 0, availability >= 0) 4. User submits request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-02	
Objective: Ensure the system will not add duplicate items to the database (existing itemID	
Set up: A The user is logged in as an administrator, all systems are working as intended	
Expected result: 1. The system receives a request to add an item to the database 2. The system identifies the itemID as being duplicate (item already exists in the database) 3. System informs the user that the item already exists in the database 4. System does NOT add item to the database	
Test: 1. User navigates to catalogue page 2. User selects to add a new item 3. User inputs item details (duplicate itemID) 4. User submits request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-03	
Objective: Ensure the system will not add items to the database with invalid values (price < 0 or availability < 0)	
Set up: The user is logged in as an administrator, all systems are working as intended	
Expected result: 1. The system receives a request to add an item to the database 2. The system identifies the input as being invalid (price < 0 or availability < 0) 3. System informs the user that the item values are invalid 4. System does NOT add item to the database	
Test: 1. User navigates to catalogue page 2. User selects to add a new item 3. User inputs item details (price < 0 or availability < 0) 4. User submits request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-04	
Objective: Ensure that an item in the database can be updated	
Set up: The user is logged in as an administrator, all systems are working as intended	
Expected result: 1. System receives an item update request 2. System finds item with the given itemID 3. System updates item in the database based on data given in update request 4. System informs the user the item has been updated	
Test: 1. User navigates to catalogue page 2. User selects to update an existing item 3. User selects target item using itemID 4. User enters data to be updated 5. User submits update request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-05	
Objective: Ensure the system can delete an item from the database	
Set up: The user is logged in as an administrator, all systems are working as intended	
Expected result: 1. System receives an item deletion request 2. System ensures the user has authorisation 3. System ensures the item is in the database 4. System deletes the item from the database 5. System informs the user that the item has been deleted	
Test: 1. User navigates to the catalogue page 2. User selects to delete an item 3. User selects target item with itemID 4. User submits item deletion request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-06	
Objective: Ensure the system can search for an item by a valid itemID	
Set up: The user is logged in as a merchant, manager, or administrator, and the system is working as intended.	
Expected result: 1. System receives a request to search by itemID 2. System searches the database for an item with the given ID 3. System returns item with the given itemID	
Test: 1. User navigates to the catalogue page 2. User selects to search for an item 3. User selects target item with itemID 4. User submits search request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-07	
Objective: Ensure the system correctly handle a search with an invalid itemID	
Set up: The user is logged in as a merchant, manager, or administrator, and the system is working as intended.	
Expected result: 1. System receives a request to search by itemID 2. System searches the database for an item with the given ID 3. System cannot locate item (invalid itemID) 4. System informs user the item was not found	
Test: 1. User navigates to the catalogue page 2. User selects to search for an item 3. User selects target item with itemID (invalid itemID) 4. User submits search request	
Test record:	
Date: 19/02/2026	**Tester:** Nikki Baker
Result:	

Test number: CAT-08	
Objective: Test the search functionality by keyword (valid search for a keyword ‘para’)	
Set up: User is logged in as an Administrator. The catalogue contains items with descriptions containing the string ‘para’	
Expected results: 1. System receives the string input ‘para’ 2. System checks the catalogue database for matches in description 3. Matching items are returned and displayed to the user 4. If multiple items match, they are displayed in a list 5. User remains on the catalogue search page	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section. 2. User selects the search bar for ‘Search by Keyword’ 3. User types ‘para’ and presses enter 4. Search request is submitted	
Test record: Expected results observed. Matching items are returned with correct details	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-09	
Objective: Test the search functionality by keyword when a search is attempted with an empty keyword	
Set up: User is logged in as an Administrator. The catalogue contains existing items	
Expected results: 1. System receives empty string input 2. System prevents the request from being sent to the database 3. User receives a validation error message or an empty result is displayed 4. User is asked to enter a valid keyword	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section. 2. User selects the search bar for ‘Search by Keyword’ 3. User doesn’t type anything and presses enter 4. Search request is submitted	
Test record: Expected results observed. Validation error triggered	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-10	
Objective: Test the valid stock addition to an existing catalogue item	
Set up: User is logged in as an Administrator. The database contains an existing item of ID 101 with a current availability of 50 packs	
Expected results: 1. System validates that the Item ID 101 exists in the database 2. System validates that the number of packs to add is greater than 3. The availability for the item increases by the specified amount. 4. User receives a confirmation message that stock has been updated	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section. 2. User selects the 'addStock' operation for a specific item. User enters: - Item ID: 101 - packsToAdd: 25 3. Request is submitted	
Test record: Expected results observed. Item 101 availability increased from 50 to 75	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-11

Objective:

Test the boundary stock addition to an existing catalogue item (attempting to add 0 packs)

Set up:

User is logged in as an Administrator. The database contains an existing item of ID 102

Expected results:

1. System validates that the Item ID 102 exists in the database

2. System validates that the number of packs to add is equal to 0

3. System prevents the update to the database

4. User receives a validation error message

5. Item availability is unchanged

Test:

1. User goes to the Catalogue (IPOS-SA-CAT) section.

2. User selects the 'addStock' operation for a specific item.

User enters:

- Item ID: 102
- packsToAdd: 0

4. Request is submitted

Test record: Expected results observed. Validation error triggered

Date: 18/02/2026	**Tester:** Mayan Kanji

Result: Passed

Test number: CAT-12	
Objective: Test the stock addition to an unknown catalogue item	
Set up: User is logged in as an Administrator. The database does not contain item of ID 103	
Expected results: 1. System attempts to locate item ID 103 in the database 2. System identifies that the Item ID is invalid/unknown 3. System displays a 'Not found error' message to the user 4. No changes are made to the database	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section 2. User selects the 'addStock' operation for a specific item User enters: - Item ID: 103 - packsToAdd: 25 4. Request is submitted	
Test record: Expected results observed. 'Not found error' message triggered	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-13	
Objective: Test the main flow of setting a valid minimum stock level for an item	
Set up: User is logged in as an Administrator. The database contains an existing item ID 104	
Expected results: 1. System validates that the minLevel is greater than or equal to 0 2. The new minimum threshold is saved to the item record 3. User receives a confirmation message that the minimum has been saved	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section 2. User selects the 'setMinStockLevel' operation for a specific item User enters: - Item ID: 104 - minLevel: 10 4. Request is submitted	
Test record: Expected results observed. Minimum stock level for Item 101 updated to 10	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-14	
Objective: Test the main flow of setting a negative minimum stock level for an item	
Set up: User is logged in as an Administrator. The database contains an existing item ID 105	
Expected results: 1. System identifies that minLevel is less than 0 2. System prevents the update to the database 3. User receives a validation error message 4. The previous minimum stock level remains unchanged	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section 2. User selects the 'setMinStockLevel' operation for a specific item User enters: - Item ID: 105 - minLevel: -2 4. Request is submitted	
Test record: Expected results observed. Validation error triggered	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-15	
Objective: Test the generation of a report for items currently below their minimum stock threshold (items below minimum)	
Set up: User is logged in as Operations manager. The database contains at least one item where the current availability is less than its minLevel	
Expected results: 1. System checks all items in the database 2. System identifies all items where availability is less than minLevel 3. System generates these items into a list 4. Report is generated and displayed to the user 5. User remains on the catalogue page	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section 2. User selects the 'listLowStock' operation 3. Request is submitted	
Test record: Expected results observed. Report correctly displayed low-stock items	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: CAT-16	
Objective: Test the generation of a report when no items are currently below their minimum stock threshold (no low stock items)	
Set up: User is logged in as Operations manager. The database is manipulated so that for every item, the current availability is greater than or equal to its minLevel	
Expected results: 1. System checks all items in the database 2. System identifies zero items where availability is less than minLevel 3 An empty list is returned to the user 4. User remains on the catalogue page	
Test: 1. User goes to the Catalogue (IPOS-SA-CAT) section 2. User selects the ‘listLowStock’ operation 3. Request is submitted	
Test record: Expected results observed. Empty list returned to the user	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-01	
Objective: Test the main flow for a valid order placement by a merchant	
Set up: User is logged in as a Merchant. The merchant's account status is NORMAL. The database contains existing item ID 105, current availability: 44	
Expected results: 1. System checks that the requested quantity is less than or equal to current availability 2. Order is created in the database 3. Item stock is reduced by the ordered quantity 4. User receives a confirmation that the order has been placed	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section. 2. User selects the ‘placeOrder’ operation User enters: - Merchant ID: MCH-001 - Item ID**:** 105 - Quantity**:** 12 3. Request is submitted	
Test record: Expected results observed. Order created successfully and stock for Item 101 was reduced to 32	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-02	
Objective: Test the main flow for an order placement where order quantity exceeds stock availability	
Set up: User is logged in as a Merchant. The database contains existing item ID 106, current availability: 3	
Expected results: 1. System checks that the requested quantity is less than or equal to current availability 2. System identifies that the requested quantity is greater than current availability 3. Order is rejected 4. User receives an error message 5. No changes are made to the items stock level in the database	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section. 2. User selects the 'placeOrder' operation User enters: - Merchant ID: MCH-001 - Item ID**:** 106 - Quantity**:** 15 3. Request is submitted	
Test record: Expected results observed. Order was rejected	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-03	
Objective: Test the main flow for an order placement where the itemID is unknown	
Set up: User is logged in as a Merchant	
Expected results: 1. System checks to see if the itemID is in the database 2. itemID is not in the database, system identifies an invalid itemID 3. Order is rejected 4. User receives an error message	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the 'placeOrder' operation User enters: - Item ID**:** 107 3. Request is submitted	
Test record: Expected results observed. Order was rejected	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-04	
Objective: Test the main flow for an empty order input	
Set up: User is logged in as a Merchant. The order contains no order lines	
Expected results: 1. System checks for a valid input entry in the order 2. System identifies that the order is empty 3. Order is rejected 4. User receives an error message asking them to add items to the order	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section. 2. User selects the ‘placeOrder’ operation 3. Request is submitted without the user entering any ItemIDs or quantities	
Test record: Expected results observed. Order was rejected	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-05	
Objective: Test the main flow for an order placement where the boundary quantity value is 0	
Set up: User is logged in as a Merchant. The database contains existing item ID 108	
Expected results: 1. System checks that the requested quantity is greater than 0 2. System identifies that the quantity is exactly 0 3. Order is rejected 4. User receives a validation error message	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the 'placeOrder' operation User enters: - Item ID**:** 108 - Quantity: 0 3. Request is submitted	
Test record: Expected results observed. Validation error occurred and order was rejected	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-06	
Objective: Test the main flow for an order placement by a merchant whose account is suspended	
Set up: User is logged in as a Merchant. Merchants account status is SUSPENDED	
Expected results: 1. System checks the merchant's status 2. System identifies that the merchant's status is SUSPENDED 3. Order is rejected 4. User receives an error message	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section. 2. User selects the 'placeOrder' operation User enters: - MerchantID: MCH-16 - Item ID**:** 109 - Quantity: 3 3. Request is submitted	
Test record: Expected results observed. Order was rejected and system displayed an error message due to suspended account	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-07	
Objective: Test the main flow for an order placement by a merchant who's flagged as 'In default'	
Set up: User is logged in as a Merchant. Merchants account status is set to 'In default'	
Expected results: 1. System checks the merchants status 2. System identifies that the merchants status is 'In default' 3. Order is rejected 4. User receives an error message	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the "placeOrder" operation User enters: - MerchantID: MCH-17 - Item ID**:** 110 - Quantity: 3 3. Request is submitted	
Test record: Expected results observed. Order was rejected and system prevented the Merchant from placing an order	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-08

Objective:

Test the main flow for updating an order status to ‘Dispatched’ with valid courier and tracking information (Valid dispatch update)

Set up:

User is logged in as an Operations staff. An existing order is currently in ‘Pending’ or ‘Confirmed’ status within the database

Expected results:

1. System validates that the order exists and is eligible for dispatch.

2. System records the courier’s name and the delivery reference number

3. The order status is updated to ‘Dispatched’

4. User receives a confirmation that the status has been successfully updated

Test:

1. User goes to the Order Service (IPOS-SA-ORD) section.

2. User selects the ‘updateOrderStatus operation

User enters:

- Order ID: 1008
- New Status**:** DISPATCHED
- Courtier: DHL
- Reference Number: REF-992211

3. Request is submitted

Test record: Expected results observed. Order 1008 status changed to ‘Dispatched’

Date: 18/02/2026	**Tester:** Mayan Kanji

Result: Passed

Test number: ORD-09

Objective:

Test the main flow for updating an order status to ‘Dispatched’ without courtier, reference and delivery details

Set up:

User is logged in as an Operations staff. An existing order is currently in ‘Pending’ or ‘Confirmed’ status within the database

Expected results:

1. System receives request and the order status is updated to ‘Dispatched’

2. System identifies that the mandatory fields (Courier, Reference, Delivery) are missing.

3. Request is rejected

4. Order status remains unchanged in the database

5. User receives an error message

Test:

1. User goes to the Order Service (IPOS-SA-ORD) section.

2. User selects the ‘updateOrderStatus operation

User enters:

- Order ID: 1009
- New Status**:** DISPATCHED
- Courtier: [Leave Blank]
- Reference Number: [Leave Blank]

3. Request is submitted

Test record: Expected results observed. System rejected status change because there is missing dispatch details

Date: 18/02/2026	**Tester:** Mayan Kanji

Result: Passed

Test number: ORD-10	
Objective: Test the main flow for a Merchant updating an order status	
Set up: User is logged in as a Merchant. An existing order is present within the database	
Expected results: 1. System identifies the user as a Merchant, this role does not have authorization to update order status 2. Access is denied 3. No changes are made to the order record in the database	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the ‘updateOrderStatus’ operation 3. User attempts to update the order status to ‘Dispatched’ 4. Request is submitted	
Test record: Expected results observed. System rejected the request and returned an ‘Access Denied’ error	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test number: ORD-11	
Objective: Test the system's ability to retrieve and display a specific order using a valid ID	
Set up: User is logged in as a Merchant. An existing order is present within the database	
Expected results: 1. System receives a valid Order ID 2. System searches the database and locates the matching record. 3. The order details are returned and displayed to the user	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the ‘getOrder’ operation 3. User enters: - Order ID: 3001 4. Request is submitted	
Test record: Expected results observed. Order 3001 was successfully returned	
Date: 18/02/2026	**Tester:** Mayan Kanji
Result: Passed	

Test Number: ORD-12	
Objective: Test the system behaviour when attempting to retrieve an order using an invalid Order ID	
Set up: User is logged in as a Merchant. The database does not contain the specified Order ID.	
Expected result: 1. System receives the invalid Order ID 2. System searches the database for a matching order 3. System determines the order does not exist 4. System displays a “Not found” error message 5. No data is returned to the user	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the ‘getOrder’ operation 3. User enters an invalid Order ID 4. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: ORD-13	
Objective: Test retrieving orders for a merchant within a valid date range	
Set up: User is logged in as a Merchant. The database contains orders associated with the merchant within the selected date range.	
Expected result: 1. System receives a valid start and end date 2. System verifies the date range is valid 3. System searches for orders belonging to the merchant within the date range 4. Matching orders are returned and displayed	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the 'listOrdersByMerchant' operation 3. User enters a valid start date and end date 4. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: ORD-14	
Objective: Test the system behaviour when an invalid date range is entered	
Set up: User is logged in as a Merchant.	
Expected result: 1. System receives the start and end dates 2. System detects that the end date is earlier than the start date 3. System rejects the request 4. System displays a validation error message	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the 'listOrdersByMerchant' operation 3. User enters an end date earlier than the start date 4. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: ORD-15	
Objective: Test retrieving the outstanding balance for a merchant account	
Set up: User is logged in as a Merchant. The merchant has an outstanding balance recorded in the database.	
Expected result: 1. System receives a valid Merchant ID 2. System retrieves the account balance from the database 3. Outstanding balance is displayed to the user	
Test: 1. User goes to the Order Service (IPOS-SA-ORD) section 2. User selects the 'getOutstandingBalance' operation 3. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: INV-01	
Objective: Test invoice generation after a valid accepted order	
Set up: An order exists in the database and has been successfully accepted by the system.	
Expected result: 1. System detects a valid accepted order 2. System generates a new invoice 3. Invoice is stored in the database 4. Invoice is linked to the corresponding order	
Test: 1. User places a valid order 2. System processes the order 3. Invoice is automatically generated	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: INV-02	
Objective: Test invoice generation for an order not in an accepted state	
Set up: An order exists but its status is PROCESSING.	
Expected result: 1. System checks order status 2. System identifies order is not accepted 3. Invoice generation is rejected 4. Error message is displayed	
Test: 1. User attempts to generate invoice for processing order 2. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: INV-03	
Objective: Test invoice generation using an invalid Order ID	
Set up: The specified Order ID does not exist in the database.	
Expected result: 1. System searches database 2. System does not find the order 3. System returns not found error 4. No invoice is created	
Test: 1. User attempts to generate invoice 2. User enters invalid Order ID 3. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: INV-04	
Objective: Test retrieving a stored invoice	
Set up: A valid invoice exists in the database.	
Expected result: 1. System receives valid Invoice ID 2. System retrieves invoice record 3. Invoice details are displayed	
Test: 1. User selects the ‘getInvoice’ operation 2. User enters valid Invoice ID 3. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: INV-05	
Objective: Test retrieving an invoice with an invalid Invoice ID	
Set up: Invoice does not exist in the database.	
Expected result: 1. System searches database 2. Invoice not found 3. Error message displayed	
Test: 1. User selects ‘getInvoice’ 2. User enters invalid Invoice ID 3. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: : INV-06	
Objective: Test retrieving invoices for a merchant within a valid date range	
Set up: Invoices exist in the database for the specified merchant.	
Expected result: 1. System validates date range 2. System retrieves matching invoices 3. List of invoices displayed	
Test: 1. User selects 'listInvoicesByMerchant' 2. User enters merchant ID and valid dates 3. Request is submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-01	
Objective: Test recording a valid payment	
Set up: Accounting staff is logged in. Merchant has outstanding balance.	
Expected result: 1. System receives payment amount greater than 0 2. Payment recorded 3. Balance updated	
Test: 1. Staff selects ‘recordPayment’ 2. Enters valid amount 3. Request submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number : PAY-02	
Objective: Test recording an invalid payment amount	
Set up: Accounting staff logged in.	
Expected result: 1. System detects amount ≤ 0 2. Request rejected 3. Validation error displayed	
Test: 1. Staff enters amount 0 2. Request submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-03	
Objective: Test merchant attempting to record a payment	
Set up: User logged in as Merchant.	
Expected result: 1. System checks user role 2. Access denied 3. No changes made	
Test: 1. Merchant selects ‘recordPayment’ 2. Request submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-04	
Objective: Test applying a recorded payment	
Set up: Valid payment exists	
Expected result: 1. System retrieves payment 2. Merchant balance reduced	
Test: 1. Staff selects ‘applyPayment’ 2. Enters payment ID 3. Request submitted	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-05	
Objective: Test payment overdue up to 15 days	
Set up: Merchant payment overdue 10 days.	
Expected result: 1. System detects overdue payment 2. Reminder displayed 3. Account remains usable	
Test: Merchant logs into account	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-06	
Objective: Test payment overdue between 15 and 30 days	
Set up: Merchant payment overdue 20 days.	
Expected result: 1. System detects overdue payment 2. Account suspended	
Test: Merchant logs in	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-07	
Objective: Test payment overdue more than 30 days	
Set up: Merchant payment overdue 35 days.	
Expected result: 1. System detects overdue payment 2. Account flagged IN_DEFAULT	
Test: Merchant logs in	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-08	
Objective: Test restoring account after valid payment	
Set up: Payment made within 30 days.	
Expected result: 1. System verifies payment 2. Account restored to NORMAL	
Test: Staff records payment	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad
Result: Passed	

Test Number: PAY-09	
Objective: Test restoring account without authorisation	
Set up: No director approval provided.	
Expected result: 1. System checks approval 2. Restoration rejected 3. Account status unchanged	
Test: Staff attempts restore	
Test record:	
Date: 20/02/2026	**Tester:** Afiifa Salad

Result: Passed

Interface Test Specifications – IPOS-SA (Team 28 – London Software House)

5.1 Purpose

The objective of these tests is to ensure that:

- All interface methods act accordingly for their respective valid inputs.
- Correct validation errors or exceptions are raised for invalid inputs.
- Boundary conditions like zero values or empty inputs are handled fittingly.
- Role access restrictions are forced.

5.2 Assumptions and Test Conditions

- Merchant statuses: NORMAL, SUSPENDED, IN_DEFAULT
- Order statuses: ACCEPTED, PROCESSING, DISPATCHED
- Discount types: FIXED, FLEXIBLE
- Stock quantities are integers ≥ 0
- Monetary values must be ≥ 0
- Role restrictions apply (Merchant , Staff , Manager , Director)

Each subsection below defines the test cases for a specific IPOS-SA service interface. All methods are tested against valid scenarios, invalid scenarios, boundary values, and access control constraints.

5.3 MerchantAccountService (IPOS-SA-ACC)

Test ID	Operation	Test Scenario	Input	Expected Result	Valid/Invalid
ACC-01	createMerchantAccount	Create merchant with valid data	Full details, creditLimit > 0, FIXED discount	Account created; status = NORMAL	Valid
ACC-02	createMerchantAccount	Missing mandatory fields	Missing contact details OR creditLimit	Validation error account not created	Invalid
ACC-03	createMerchantAccount	Invalid discount rate	FIXED rate < 0 or > 100%	Validation error	Invalid
ACC-04	createMerchantAccount	Boundary credit limit	creditLimit = 0	Validation error	Invalid
ACC-05	updateCreditLimit	Valid update	existing merchantId, newLimit > 0	Credit limit updated	Valid
ACC-06	updateCreditLimit	Negative limit	newLimit < 0	Validation error	Invalid
ACC-07	updateDiscountPlan	Valid switch	valid FLEXIBLE thresholds	Plan updated	Valid
ACC-08	updateDiscountPlan	Null plan	discountPlan = null	Validation error	Invalid

ACC-09	setMerchantStatus	Suspend merchant	merchantId, SUSPENDED by Manager	Status updated	Valid
ACC-10	setMerchantStatus	Unauthorised status change	Non-manager role	Access denied	Invalid
ACC-11	setMerchantStatus	Restore IN_DEFAULT without approval	No director authorisation	Reject	Invalid
ACC-12	getMerchant	Retrieve valid merchant	valid merchantId	Merchant returned	Valid
ACC-13	getMerchant	Unknown merchant	invalid merchantId	Not found error	Invalid

5.4 CatalogueService (IPOS-SA-CAT)

Test ID	Operation	Test Scenario	Input	Expected Result	Valid/Invalid
CAT-01	addItem	Add valid item	unique itemId, price ≥ 0, availability ≥ 0	Item created	Valid
CAT-02	addItem	Duplicate item ID	existing itemId	Reject duplicate	Invalid
CAT-03	addItem	Invalid values	price < 0 OR availability < 0	Validation error	Invalid
CAT-04	updateItem	Update valid item	existing itemId	Item updated	Valid
CAT-05	deleteItem	Delete valid item	existing itemId	Item removed	Valid
CAT-06	searchById	Search valid item	valid itemId	Item returned	Valid
CAT-07	searchById	Search unknown item	invalid itemId	Not found error	Invalid
CAT-08	searchByKeyword	Valid keyword	"para"	Matching items returned	Valid
CAT-09	searchByKeyword	Empty keyword	""	Validation error or empty result	Invalid

CAT-10	addStock	Valid stock addition	packsToAdd > 0	Availability increases	Valid
CAT-11	addStock	Boundary stock	packsToAdd = 0	Validation error	Invalid
CAT-12	addStock	Unknown item	invalid itemId	Not found error	Invalid
CAT-13	setMinStockLevel	Valid minimum	minLevel ≥ 0	Minimum saved	Valid
CAT-14	setMinStockLevel	Negative minimum	minLevel < 0	Validation error	Invalid
CAT-15	listLowStock	Items below minimum	below threshold items exist	Correct report generated	Valid
CAT-16	listLowStock	No low stock items	none below threshold	Empty list returned	Valid

5.5 OrderService (IPOS-SA-ORD)

Test ID	Operation	Test Scenario	Input	Expected Result	Valid/Invalid
ORD-01	placeOrder	Valid merchant order	NORMAL merchant, qty ≤ availability	Order created; stock reduced	Valid
ORD-02	placeOrder	Quantity exceeds stock	qty > availability	Order rejected	Invalid
ORD-03	placeOrder	Unknown item ID	invalid itemId	Order rejected	Invalid
ORD-04	placeOrder	Empty order	no order lines	Reject	Invalid
ORD-05	placeOrder	Boundary quantity	qty = 0	Validation error	Invalid
ORD-06	placeOrder	Suspended merchant	SUSPENDED merchant	Reject	Invalid
ORD-07	placeOrder	In default merchant	IN_DEFAULT merchant	Reject	Invalid
ORD-08	updateOrderStatus	Valid dispatch update	DISPATCHED + courier/ref/delivery	Status updated	Valid
ORD-09	updateOrderStatus	Missing dispatch details	DISPATCHED without details	Reject	Invalid
ORD-10	updateOrderStatus	Merchant attempts update	merchant role	Access denied	Invalid
ORD-11	getOrder	Retrieve valid order	valid orderId	Order returned	Valid

ORD-12	getOrder	Unknown order	invalid orderId	Not found error	Invalid
ORD-13	listOrdersByMerchant	Valid date range	start ≤ end	Orders returned	Valid
ORD-14	listOrdersByMerchant	Invalid date range	end < start	Validation error	Invalid
ORD-15	getOutstandingBalance	Retrieve balance	valid merchantId	Correct balance returned	Valid

5.6 InvoiceService

Test ID	Operation	Test Scenario	Input	Expected Result	Valid/Invalid
INV-01	generateInvoice	Valid accepted order	valid orderId	Invoice created and linked	Valid
INV-02	generateInvoice	Order not in accepted state	order in PROCESSING	Reject	Invalid
INV-03	generateInvoice	Unknown order	invalid orderId	Not found error	Invalid
INV-04	getInvoice	Retrieve invoice	valid invoiceId	Invoice returned	Valid
INV-05	getInvoice	Unknown invoice	invalid invoiceId	Not found error	Invalid
INV-06	listInvoicesByMerchant	Valid date range	merchantId + valid dates	Correct list returned	Valid

5.7 PaymentEntryService

Test ID	Operation	Test Scenario	Input	Expected Result	Valid/Invalid
PAY-01	recordPayment	Valid payment	amount > 0	Payment stored	Valid
PAY-02	recordPayment	Invalid amount	amount ≤ 0	Validation error	Invalid
PAY-03	recordPayment	Merchant attempts payment entry	merchant role	Access denied	Invalid
PAY-04	applyPayment	Apply payment	valid paymentId	Balance reduced	Valid

PAY-05	**evaluateStatus**	**Late ≤ 15 days**	**1–15 days overdue**	**Account usable; reminder shown**	**Valid**
PAY-06	**evaluateStatus**	**Late 15–30 days**	**16–30 days overdue**	**Account becomes SUSPENDED**	**Valid**
PAY-07	**evaluateStatus**	**Late > 30 days**	**>30 days overdue**	**Account becomes IN_DEFAULT**	**Valid**
PAY-08	**restore**	**Valid restoration**	**payment within 30 days**	**Status returns to NORMAL**	**Valid**
PAY-09	**restore**	**Restore without authorisation**	**no approval flag**	**Reject**	**Invalid**

8. Appendices

8.1 Initial Statement of requirements by the client

Initial Requirement

The InfoPharma Ordering System (IPOS) will keep track of the orders of pharmaceutical goods made by merchants to InfoPharma Ltd and will assist the merchants' sales to consumers. IPOS includes three subsystems:

- i) A server application (IPOS-SA), which in the fully functional production version will be administered by the IT department of InfoPharma Ltd.
- ii) A client application (IPOS-CA), a copy of which will be run by the merchant on a desktop PC in the merchant's Pharmacy shop.
- iii) An online platform (IPOS - PU) to be used by a merchant for direct sales of pharmaceutical goods to the public. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

In the prototype of IPOS the three subsystems should be implemented as separate desktop applications able to interact with one another as necessary to meet the requirements. For instance, online sales (via IPOS - PU) should be linked to the inventory of the merchant: successful online purchases should be deducted from the stock of goods available at merchant's shop. Likewise, orders from IPOS-SA and delivered to a merchant, should be easy to record in IPOS-CA and increase the merchant's stock. Finally, promotion campaigns run via IPOS – PU may affect the cost of selling goods online (at reduced price), but not the cost of the same goods if sold in the pharmacy shops

Noun and Verb analysis conducted by Elizabeth

noun
verb
list of fields
Apendixes
Easy to skip words

This subsystem should allow InfoPharma Ltd. to maintain an electronic catalogue of pharmaceutical products and control stock availability. The details of an item in the catalogue (products ID, product description, unit price and current availability) are detailed in Appendix 1. The catalogue is accessed by merchants who should have valid user accounts set for them by the administrator of IPOS-SA. A valid user can access the catalogue and place an order by filling in an order form (Appendix 2). Once an order is submitted IPOS-SA should generate an invoice which will be stored in the system database for further reference and can be displayed on screen or printed out by either the corresponding merchant or by the Operation's Manager at InfoPharma Ltd. The status of the orders (accepted, being processed, dispatched) will be maintained by IPOS – SA and members of staff at InfoPharma should be able to update it when needed (e.g., when an order is dispatched a member of the dispatching department will have to enter the following details: who dispatched it, when was the order dispatched, the courier used for the delivery, courier's Ref. No, expected delivery time). The merchant should be able to track their orders through IPOS, too. The merchants are given credit by InfoPharma Ltd. and can make payments for the goods they have ordered by the *end of the calendar month*. They can check their balance by accessing IPOS_SA. Payments by merchants are not part of IPOS-SA and should be made using other means, such as direct bank transfers using InfoPharma IBAN number shared with all registered merchants. Once a payment is received, payment details should be entered into the system by the accounting department at InfoPharma, and the account balance of the respective merchant - will be updated. If a merchant is late no more than 15 days with their payment, they can continue to use the ordering system, but IPOS-SA will generate a reminder on the screen every time the account is accessed by the merchant. If a merchant is late with a payment between 15 days and 30 days, then IPOS-SA will suspend the user account, i.e. no further orders will be accepted. Once a payment is received within 30 days of the payment deadline the 'suspended' account is restored to 'normal' by the system and the user can resume placing new orders. If the payment delay is longer than 30 days, the user merchant account is flagged as 'in default'. Restoring an account from an 'in default' to

‘normal’ (or ‘suspended’) state can only be done after an authorisation by the Director of operations of InfoPharma Ltd.

The functionality of IPOS-SA can be divided into the following packages:

IPOS-SA-ACC: This package provides the functionality needed for setting up accounts for the *users* of IPOS-SA.
Different users may have different privileges and use different parts of IPOS-SA. At least the following three groups of accounts must be implemented in the prototype:

i) a **merchant’s account**. The user with such an is account should be able to access the following packages: IPOS_
SA-CAT and IPOS-SA-ORD (see below). Once a new account is set up as a merchant account the system will ask for contact details to be provided, credit limit and discount plan to be set up for the new merchant before the account is activated. If the required details are not provided the account will not be created. The discount plan sets discount rates which will be applied to orders for respective merchant. At least the following two types of discount plans must be implemented:

- *fixed discount plan*, i.e. the same discount rate applies to all items of any quantity ordered from
the catalogue. The discount is applied to each order and the amount due is reduced by the amount of the discount at the time of placing an order.
- *flexible discount plan*, in which the discount rates depend on the value of all orders *within a calendar month*, e.g. 1% for orders less than £1000, 2% for orders between £1000 and £2000 and 3% for orders in excess of £2000. With the flexible discount the value of the discount is calculated at the end of the calendar month. The amount of the discount is either paid back to the merchant by sending a cheque at the end of the month or deducted from the value of the next order(s) by the same merchant.

ii) an **administrator’s account**. This account has full access to the functionality of IPOS-SA, i.e. access to all packages of IPOS-SA including IPOS-SA-ACC.
iii) a **manager’s account**. This account allows the user an access to IPOS-SA-RPRT. This user can also access the merchant accounts and alter their credit limits, discount plans and change the state of an ‘in default’ account to either ‘normal’ or ‘suspended’.

IPOS-SA-CAT:

This package contains the functionality necessary to maintain a catalogue of goods available to the merchants to order from. As a minimum, the following functions are required:

- creating a (new) catalogue (optional, if it does not already exist. It is acceptable not to have this function as part of IPOS-SA, in which case the catalogue should be created using tools available for administering the IPOS-SA database),
- adding a new item, deleting an item, updating an item, searching for an item using various searching criteria (item ID, key term, etc.),
- adding new stock of a catalogue item (e.g. when they are delivered to InfoPharma). For each item in the catalogue, a minimum stock level parameter can be set up (a non-negative integer).

The system must warn the relevant users (administrator and the operation’s manager) every time they use IPOS-SA about the occurrence of low stock conditions. A report should be possible to generate on demand to list all items in the catalogue, which are below their set minimum stock levels (see Appendix 3).

IPOS-SA-ORD:

This package contains the functionality necessary for a merchant to be able to place a new order (choosing items from the current catalogue) and track the status of their orders, the outstanding unpaid balances, etc. Once a new order is accepted by the system, the InfoPharma stock should be reduced accordingly. Raising invoices against accepted orders and entering data about the payments made for previous orders (bank transfers, card or cheque payments) is also part of this package.

IPOS-SA-RPT: Report generating package. This package should allow various reports to be generated by searching the system database. The report should be visualised on computer screen and printed (when required). The prototype should allow for generating the following reports:

i) Turnover for a *given period of time* in terms of quantities of goods sold to merchants and the revenue received by InfoPharma Ltd.

ii) List of orders received from a particular merchant for a given period of time with the following details (see Appendix 4):

- order ID,
- date of ordering,
- value of the order,
- date of dispatching,
- payment received (or pending),
- a 'Totals' line should conclude the report.

iii) Report about the activity of an individual merchant for a given period of time with the following details (see Appendix 5):

- contact details of a merchant (as a header),
- list of all orders (including the individual items ordered, quantity ordered, individual cost, total cost of order, discount given if known, payment status).

iv) List of invoices raised against an individual merchant for a given period (on the screen) and then navigating through the details of the individual invoices. Printouts of all these should also be possible to generate.

v) List of all invoices raised by InfoPharma Ltd against merchants for a given period.

vi) Stock turnover (goods sold and newly received) within a given period of time.

---

[9] In the full production version of IPOS payment may be integrated with on-line banking, i.e. the merchants may be able to make payment by bank transfers. In the prototype, however, the payment details are expected to be entered into the system manually

8.1.2 Hardware (minimal and optimal configuration)

8.1.2.1 Minimal hardware configuration

The minimal hardware configuration, recommended for IPOS-SA development consists of a desktop PC, 1 GB of RAM and at least 50 GB of disk space for storing the software programs and data, possibly running a database server such as MySQL. A hosting package with preinstalled software and at least 1 GB for use data is also acceptable for both development and live deployment of IPOS-SA.

8.1.2.2 Optimal hardware configuration

The optimal hardware configuration envisaged for development should consist of a desktop PC, 4 GB of RAM and at least 250 GB of disk space for storing the software programs and data, possibly running a database server such as MySQL or better. Again, a hosting package with preinstalled software + 5 GB of data is considered an optimal hardware configuration for both development and live deployment.

9. Credit List

The list below clearly lists which components of this document were worked on by which team members. This is for ease of review, to allow each team member's contributions to be seen and weighed fairly, often in other pieces of work, which were then collated. However, for a more extensive check, the version history can be consulted.

Member:	Contribution to Document:
Elizabeth	Wrote the main part of the Requirements Specification and High-Level System Design document. Did the noun verb Analysis (see section 8.1 at the bottom) and prioritised list of use cases.
Humd	Planned, created and refined every Use Case Specification, Completed Section 8.1.1/1.2 on appropriate hardware, formatting of the document title page and including page number, company logo and this table for efficiency in marking. Final Checks and formatting.
Ali	Created the Test Case Specifications
Mayan	Use case Diagram and Test Cases 21-40
Afifa	Test cases 41-60
Nizar	Planned for the design and programmer roles,Component,Class Diagrams and ALL of the java classes.
Nikki	Class Diagram and initial structure skeleton of the java classes. Test cases 1-20, development of class diagram. Generating base java interface code from VP project